KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 07

Miras (Inheritance)

`extends` ve `super` · yapıcı zinciri · metot ezme ve `@Override` · `protected` ve `final` · `Object` kökü · IS-A / HAS-A kararı · elmas problemi

ÖZET

Hafta 2'den bu yana yazdığımız her sınıf yalnız başınaydı. `Ogrenci` ve `Personel` sınıflarının ikisinde de `ad`, `tcNo` ve `dogumYili` alanları ile `yas()` metodu birebir tekrarlandı; Hafta 3'te bunun bir çözümü olduğunu söylemiş, ayrıntıyı bu haftaya bırakmıştık. Bu hafta o borcu ödüyoruz. Miras, iki sınıf arasında **tür akrabalığı** kurar: "Öğrenci bir kişidir" cümlesi koda çevrilir ve ortak payda tek bir üst sınıfta toplanır. Haftanın sonunda `extends` ile sınıf türetebilecek, `super` sözcüğünün üç ayrı kullanımını ayırt edebilecek, yapıcı zincirinin neden en tepeden aşağıya doğru çalıştığını gösterebilecek ve en önemlisi **kalıtım ile bileşim arasında gerekçeli karar** verebileceksiniz. Bölüm 4.7'de Java'nın çoklu sınıf mirasını neden yasakladığını (elmas problemi) inceleyecek, Bölüm 4.6'da her sınıfın gizli atası olan `Object` sınıfıyla tanışacağız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası 3

1.1. Ön Koşul Bilgi Kontrolü 4

1.2. Haftanın Zaman Planı 4

2. Motivasyon: Aynı Kodu İki Kez Yazmanın Bedeli 5

2.1. Önceki Haftaların Bıraktığı Durum 5

2.2. Mirasla Yazılmış Hâli 6

3. Kavramın Tanımı ve Sözdizimi 8

3.1. Sözdizimi ve Sınıf Hiyerarşisi 8

3.2. Alt Sınıf Neyi Devralır, Neyi Devralmaz? 9

3.3. Bir Alt Sınıfın Anatomisi 10

3.4. Nesne Bellekte Nasıl Yerleşir? 11

3.5. IS-A İlişkisinin Sınanması: `instanceof` 12

4. Derinleşme 13

4.1. Yapıcı Zinciri ve `super(...)` 13

4.2. Metot Ezme (Override) ve `@Override` 16

4.3. `super` Sözcüğünün Üç Kullanımı 17

4.4. `protected`: Mirasla Anlam Kazanan Erişim Düzeyi 19

4.5. `final`: Mirasın Kapısını Kapatmak 20

4.6. Her Sınıfın Kökü: `Object` 22

4.7. Çoklu Miras ve Elmas Problemi 24

4.8. IS-A mı HAS-A mı? Kalıtım ile Bileşim Arasındaki Karar 25

5. C'den Java'ya Köprü 28

6. Sık Yapılan Hatalar 30

6.1. Alan Gölgeleme (Field Hiding) 30

6.2. Ezme Yerine Aşırı Yükleme 31

6.3. Yapıcıdan Ezilebilir Metot Çağırmak 32

6.4. Kalıtımın Kötüye Kullanımı 33

6.5. Derleyici İletileri Sözlüğü 33

7. İyi Tasarım İlkeleri 34

8. Bir Sonraki Haftaya Köprü 35

9. Haftanın Özeti 35

10. Alıştırmalar 36

10.1. Kavram Soruları 36

10.2. Kod Okuma ve Hata Bulma 37

10.3. Programlama Alıştırmaları 38

11. Öz Değerlendirme 39

Öz Değerlendirme Kontrol Listesi 39

12. Kaynaklar 40

12.1. Ders Kitapları 40
12.2. Çevrimiçi Kaynaklar 40
12.3. İletişim ve Destek.....40

1. Bu Haftanın Yol Haritası

Altı haftadır sınıf yazıyoruz. Hafta 2'de sınıfın ne olduğunu, Hafta 4'te alanları nasıl koruyacağımızı, Hafta 5'te nesnenin nasıl doğduğunu, Hafta 6'da bir üyenin sınıfa mı nesneye mi ait olduğunu öğrendik. Bütün bu haftalarda ortak bir varsayım vardı: *her sınıf kendi başına, diğerlerinden bağımsız bir birimdir.*

Bu varsayım gerçek sistemlerde çabuk çöker. Bir okul yönetim sisteminde `Ogrenci` ve `Personel` sınıflarını yan yana yazdığınızda ikisinin de bir adı, bir T.C. kimlik numarası ve bir doğum yılı olduğunu görürsünüz. Yaş hesabı ikisinde de aynıdır. Kimlik maskeleme ikisinde de aynıdır. Bunlar kopyalandığında ortaya çıkan sorun yalnızca "fazla satır yazmak" değildir: **iki kopya zamanla birbirinden ayrışır.** Birinde düzeltilen bir hata diğerinde kalır ve bu tür hatalar derleyici tarafından asla yakalanmaz.

Java'nın bu soruna yanıtı **miras**tır (inheritance). Miras, bir sınıfın başka bir sınıfın alanlarını ve metotlarını devralmasını sağlar. Ancak mirasın asıl değeri kod tekrarını azaltması değildir; asıl değeri **tür akrabalığı** kurmasıdır. Bir `Ogrenci` nesnesi, miras sayesinde `Kisi` beklenen her yerde kullanılabilir hâle gelir. Bu yetenek Hafta 9'da göreceğimiz çok biçimliliğin temelidir.

Bu haftanın teknik yükü orta düzeydedir: iki anahtar sözcük (`extends`, `super`), bir açıklama etiketi (`@Override`) ve bir erişim düzeyi (`protected`). Asıl zorluk yine **karar vermededir**: iki sınıf arasında miras kurmak mı, birini diğerinin içine alan olarak yerleştirmek mi doğrudur? Yanlış verilen bu karar, düzeltmesi en pahalı tasarım hatalarından biridir.

◎ Bu Bölümün Sonunda Öğrenci

- Bir sınıfı `extends` ile başka bir sınıftan türetebilir; alt sınıfın üst sınıftan hangi üyeleri devraldığını ve hangilerini devralmadığını söyleyebilir.
- "X bir Y'dir" (IS-A) ile "X'in bir Y'si vardır" (HAS-A) ilişkilerini ayırt edebilir ve bir tasarım kararını bu ayrıma dayanarak **gerekçelendirebilir**.
- `super(...)` çağrısının niçin yapıcının ilk deyimi olmak zorunda olduğunu açıklayabilir; yapıcı zincirinin en tepeden aşağıya doğru çalıştığını çalışan bir çıktı üzerinde gösterebilir.
- Bir metodu ezebilir (override), `@Override` etiketinin hangi hatayı yakaladığını açıklayabilir ve `super.metot()` ile ezilen üst sürümü yeniden kullanabilir.
- Metot ezme ile aşırı yükleme arasındaki farkı imza düzeyinde açıklayabilir; yanlışlıkla aşırı yüklenmiş bir metodu tespit edip düzeltebilir.
- `protected` erişim düzeyinin ne zaman gerektiğini, `private` ile `public` arasındaki yerini ve getirdiği sözleşme yükünü değerlendirebilir.
- `final` sınıf ve `final` metodun mirası nasıl kısıtladığını açıklayabilir; bir sınıfı ne zaman `final` yapmak gerektiğine karar verebilir.
- Her sınıfın `Object` sınıfından türediğini kanıtlayabilir; `toString()` ve `equals()` metotlarının varsayılan davranışını açıklayabilir.
- Java'nın çoklu sınıf mirasını neden yasakladığını **elmas problemi** üzerinden anlatabilir ve çoklu arayüz gerçeklemesinin bu sorunu nasıl aştığını özetleyebilir.
- Alan gölgeleme, yanlışlıkla aşırı yükleme ve yapıcıdan ezilebilir metot çağırma hatalarını tanıyabilir; üçünün de derleme hatası vermediğini bilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamını Hafta 1–6 bilgisiyle yanıtlayabiliyor olmalısınız. Yanıtlayamadığınız madde varsa laboratuvar öncesinde ilgili bölümü tekrar etmeniz önerilir.

1. Hafta 4'te öğrendiğimiz dört erişim düzeyi nelerdi? Hangisini o hafta "şimdilik erteliyoruz" diyerek atlamıştık? (Hafta 4, Bölüm 3)
2. `new Ogrenci("Elif", 20)` çağrısı çalıştığında nesne başlatma sırası nedir? Alan başlatıcıları, başlatma bloğu ve yapıcı gövdesi hangi sırayla çalışır? (Hafta 5, Bölüm 4.1)
3. Aşırı yükleme (overloading) nedir? İki metodun aşırı yüklenmiş sayılabilmesi için nelerin farklı olması gerekir, nelerin farklı olması **yetmez**? (Hafta 5, Bölüm 3.2)
4. `static` bir alan ile örnek alanı arasındaki fark nedir? Bir sınıfın `static` üyesi kaç kopya hâlinde bulunur? (Hafta 6, Bölüm 3.1)
5. `final` bir alan kaç kez atanabilir? `final` bir referans alanının gösterdiği nesnenin içeriği değişebilir mi? (Hafta 6, Bölüm 4.5)
6. Hafta 3'te dört temel ilkeyi tanıtırken "miras" için verilen minimal örnek neydi? O örnekte hangi ayrıntıyı bu haftaya bırakmıştık? (Hafta 3, Bölüm 8)

1.2. Haftanın Zaman Planı

Oturum	Süre	İçerik	Çıktı
Teori 1	50 dk	Motivasyon (Bölüm 2), `extends` sözdizimi, IS-A ilişkisi, miras alınan üyeler (Bölüm 3)	İki seviyeli bir hiyerarşiyi okuyup açıklayabilme
Teori 2	50 dk	Yapıcı zinciri, metot ezme, `super`'in üç kullanımı, `protected`, `final`, `Object`, elmas problemi, IS-A/HAS-A kararı (Bölüm 4–7)	Kalıtım–bileşim kararını gerekçelendirebilme
Lab 1	50 dk	Görev 0–2: ortam doğrulaması, `Yayin → Kitap` hiyerarşisi, `super` ile yapıcı zinciri, üç seviyeli hiyerarşi	Çalışan iki ve üç seviyeli hiyerarşiler
Lab 2	50 dk	Görev 3–5: ortak davranışı üst sınıfa taşıma, IS-A/HAS-A kararı, kalıtımı bileşime dönüştürme (bonus)	Teslim: `Hafta07_<Numara>_<AdSoyad>.zip`

Tablo 7.1 — Hafta 07 oturum planı ve beklenen çıktılar

> **ⓘ BİLGİ**
>
> Bu haftanın bütün örnekleri Hafta 2–6'da yazdığımız `Ogrenci`, `Personel` ve `Hesap` sınıflarının **doğrudan devamıdır**. Laboratuvar ise kütüphane alanına geçerek `Yayin`, `Kitap` ve `Dergi` sınıflarıyla aynı kavramları bağımsız olarak uygulatacaktır.

2. Motivasyon: Aynı Kodu İki Kez Yazmanın Bedeli

Somut bir durumla başlayalım. Okul yönetim sistemimizde iki tür insan var: öğrenciler ve personel. Hafta 4 ve 5'te öğrendiklerimizle bu iki sınıfı yazdığımızda ortaya çıkan tablo şudur.

2.1. Önceki Haftaların Bıraktığı Durum

onceki_durum.java (öğretici özet)

```java
class Ogrenci {
    private final String ad;
    private final String tcNo;
    private final int    dogumYili;
    private final String ogrenciNo;
    private final String bolum;

    public int yas() {
        return 2026 - dogumYili;
    }
    public String maskeliTcNo() {
        return "*********" + tcNo.substring(9);
    }
    public boolean resitMi() {
        return yas() >= 18;
    }
    // ... ogrenciye ozgu metotlar
}

class Personel {
    private final String ad;
    private final String tcNo;
    private final int    dogumYili;
    private final String sicilNo;
    private String unvan;

    public int yas() {
        return 2026 - dogumYili;
    }
    public String maskeliTcNo() {
        return "*********" + tcNo.substring(9);
    }
    public boolean resitMi() {
        return yas() >= 18;
    }
    // ... personele ozgu metotlar
}
```

Kod 7.1 — Hafta 4–5 bilgisiyle yazılan iki bağımsız sınıf (özet)

İki sınıfta **on iki satır birebir aynıdır**: üç alan bildirimi ve üç metot. Bu tekrarın görünür bedeli fazla yazmaktır; görünmeyen bedeli ise aşağıdaki tablodadır.

Olay	İki bağımsız sınıfta sonuç	Tek üst sınıfta sonuç
Yaş hesabı değişti (yıl sabiti sistem saatinden alınacak)	İki dosya düzeltilir; biri atlanırsa iki farklı yaş hesabı oluşur	Tek dosya düzeltilir; ikisi de otomatik doğrulanır
T.C. maskeleme kuralı değişti (son 2 → son 4 hane)	İki dosya; tutarsızlık derleme hatası vermez	Tek dosya
Üçüncü kişi türü eklenecek (`Ziyaretci`)	12 satır üçüncü kez kopyalanır	`extends Kisi` ile 3–4 satır
Bir metot listesine hem öğrenci	Ortak tür olmadığı için **imkânsız**; iki ayrı dizi	`Kisi[]` dizisi ikisini de tutar

Olay	İki bağımsız sınıfta sonuç	Tek üst sınıfta sonuç
hem personel konacak	gerekir	
“Reşit olmayan herkesi listele” istendi	İki döngü, iki ayrı kod yolu	Tek döngü, tek kod yolu

Tablo 7.2 — Kod tekrarının görünen ve görünmeyen bedelleri

Tablonun son iki satırı kritiktir. Kod tekrarı bir **bakım** sorunudur ve dikkatle yönetilebilir; ama ortak tür olmaması bir **yetenek** sorunudur. `Ogrenci` ile `Personel` arasında hiçbir tür akrabalığı olmadığı için ikisini aynı dizide tutamaz, aynı metoda parametre olarak geçiremezsiniz. Miras bu iki sorunu bir arada çözer.



Şekil 7.1 — Aynı problemin iki çözümü: bağımsız sınıflar ve ortak üst sınıf

2.2. Mirasla Yazılmış Hâli

Ortak paydayı `Kisi` adlı bir üst sınıfa taşıdığımızda `Ogrenci` sınıfının tamamı aşağıdaki gibi kısalır. Dikkat edilecek nokta: `yas()`, `resitMi()` ve `maskeliTcNo()` metotları bu dosyada **hiç yazılmamıştır**; yine de `Ogrenci` nesneleri üzerinde çağrılabilir.

Hafta_07/Java_Kodlari/ders/Ogrenci.java

```java
public class Ogrenci extends Kisi {

    // Ogrenci'ye OZGU durum: Kisi'de bunlarin karsiligi yoktur.
    private final String ogrenciNo;
    private final String bolum;
    private double gno;

    private static int uretilen = 0;   // Hafta 6'dan: statik sayac

    public Ogrenci(String ad, String tcNo, int dogumYili,
                   String ogrenciNo, String bolum) {
        // super(...) ILK satir olmak ZORUNDADIR.
```

```
13          super(ad, tcNo, dogumYili);
14          System.out.println("    [Ogrenci yapicisi calisti]");
15          this.ogrenciNo = ogrenciNo;
16          this.bolum     = bolum;
17          this.gno       = 0.0;
18          uretilen++;
19      }
20
21      public String getOgrenciNo() { return ogrenciNo; }
22      public String getBolum()     { return bolum; }
23      public double getGno()       { return gno; }
```

Kod 7.2 — `Ogrenci` sınıfının mirasla yazılmış hâli (ilk bölümü)

Bu listelemede iki yeni şey var. Birincisi 1. satırdaki `extends Kisi` bildirimi; ikincisi 13. satırdaki `super(...)` çağrısı. İkisini de Bölüm 3'te ayrıntılı inceleyeceğiz. Şimdilik sonucu görelim: `KisiDemo` programının üçüncü bölümü, `Ogrenci` sınıfında yazılmamış metotların çağrılabildiğini gösteriyor.

```
▸ TERMINAL — java KisiDemo (3. ve 4. bölüm)
=== 3) Miras alinan uyeler dogrudan kullanilir ===
  o.getAd()       -> Elif Demir
  o.yas()         -> 21
  o.resitMi()     -> true
  o.maskeliTcNo() -> *********02
  Kisi.SIMDIKI_YIL -> 2026

=== 4) Ogrenci'ye ozgu uyeler ===
  gno=3.24  basarili=true
  UYARI: GNO 0.0-4.0 araliginda olmalidir.
```

> **✓ İPUCU**
>
> Bu çıktıda `o.yas()` çağrısının çalışması mirasın en somut kanıtıdır: `Ogrenci.java` dosyasında `yas` sözcüğü hiç geçmemesine rağmen metot çağrılabilmiştir. Alt sınıf, üst sınıfın `public` ve `protected` üyelerini **kendi üyesi gibi** kullanır.

> **▲ DİKKAT**
>
> Mirası “kod tekrarından kurtulma aracı” olarak öğrenmek en yaygın kavram yanılgısıdır. İki sınıfta benzer kod bulunması tek başına miras için **gerekçe değildir**. Gerekçe, aralarında gerçek bir tür akrabalığı bulunmasıdır. Bölüm 4.8 ve 6.4 bu ayrımı ayrıntılı ele alır.

3. Kavramın Tanımı ve Sözdizimi

Miras (inheritance) Bir sınıfın (**alt sınıf**, subclass) başka bir sınıfın (**üst sınıf**, superclass) alanlarını ve metotlarını devralarak tanımlanmasıdır. Alt sınıf, devraldığı üyelerin üzerine kendi üyelerini ekler ve gerektiğinde devraldığı davranışı değiştirir.

IS-A ilişkisi Miras ile kurulan tür akrabalığıdır. `Ogrenci extends Kisi` yazıldığında "her `Ogrenci` **bir** `Kisi`dir" önermesi derleyici tarafından bilinir hâle gelir; bunun tersi doğru değildir.

Metot ezme (overriding) Alt sınıfın, üst sınıftan devraldığı bir metodu **aynı imzayla** yeniden tanımlayarak davranışını değiştirmesidir.

3.1. Sözdizimi ve Sınıf Hiyerarşisi

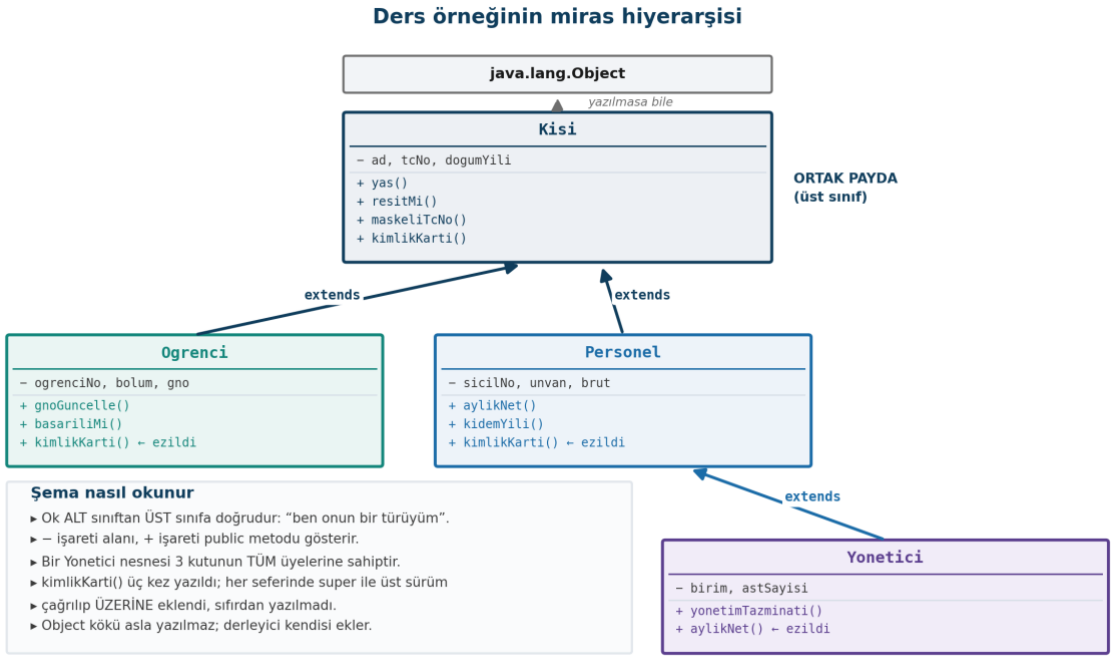
Miras `extends` anahtar sözcüğü ile kurulur. Bildirim biçimi tek satırdır ve virgülle ikinci bir sınıf yazılamaz:

sozdizimi.java (öğretici özet)

```
1  // Genel bicim
2  class AltSinif extends UstSinif { ... }
3
4  // Ders orneginden
5  public class Ogrenci  extends Kisi     { ... }
6  public class Personel extends Kisi     { ... }
7  public class Yonetici extends Personel { ... }
8
9  // YASAK: coklu sinif mirasi (Bolum 4.7)
10 // class X extends A, B { }
```

Kod 7.3 — `extends` bildiriminin biçimi

Bu üç bildirim birlikte bir **sınıf hiyerarşisi** oluşturur. `Yonetici` sınıfı hem `Personel`in hem `Kisi`nin tüm `public` üyelerine sahiptir; miras zincirlenebilir. Aşağıdaki şema ders boyunca kullanacağımız hiyerarşiyi UML sınıf diyagramına benzer bir biçimde göstermektedir.



Şekil 7.2 — Ders örneğinin miras hiyerarşisi ve şemanın okunuşu

Terim	Eş anlamlıları	Örnekte karşılığı

Terim	Eş anlamlıları	Örnekte karşılığı
Üst sınıf	superclass, temel sınıf (base class), ana sınıf	`Kisi` · `Personel` (Yonetici'ye göre)
Alt sınıf	subclass, türetilmiş sınıf (derived class)	`Ogrenci` · `Personel` · `Yonetici`
Doğrudan üst sınıf	direct superclass	`Yonetici` için yalnızca `Personel`
Dolaylı üst sınıf	indirect superclass	`Yonetici` için `Kisi` ve `Object`
Kök sınıf	root class	`java.lang.Object` (Bölüm 4.6)

Tablo 7.3 — Miras terminolojisi ve örnekteki karşılıkları

3.2. Alt Sınıf Neyi Devralır, Neyi Devralmaz?

Miras her şeyi kopyalamaz. Aşağıdaki tablo alt sınıfın üst sınıftan ne aldığını, neyi almadığını ve neyi **görmediğini** ayırır. Bu üçlü ayrım Hafta 4'teki erişim düzeyleriyle doğrudan bağlantılıdır.

Üst sınıftaki üye	Alt sınıfa geçer mi?	Açıklama
`public` alan / metot	Evet, tam erişimle	Alt sınıfın kendi üyesi gibi kullanılır
`protected` alan / metot	Evet, tam erişimle	Bu haftanın yeni erişim düzeyi (Bölüm 4.4)
Paket düzeyi (belirtilmemiş)	Aynı paketteyse evet	Farklı pakette alt sınıf göremez
`private` alan / metot	Nesnede **vardır**, ama görünmez	Bellekte yer alır; alt sınıf yalnızca getter ile erişebilir
`static` üye	Evet, ama tek kopya olarak	Alt sınıf adıyla da çağrılabilir; yeni kopya oluşmaz
Yapıcı (constructor)	**Hayır**	Yapıcı miras alınmaz; `super(...)` ile çağrılır (Bölüm 4.1)
`final` metot	Evet, ama ezilemez	Devralınır; davranışı değiştirilemez (Bölüm 4.5)

Tablo 7.4 — Mirasla geçen ve geçmeyen üyeler

✕ SIK YAPILAN HATA

En sık karşılaşılan yanılgı: "`private` alanlar miras alınmadığı için alt sınıf nesnesinde yok." Bu **yanlıştır**. `private` alan alt sınıf nesnesinin belleğinde vardır ve değerini taşır; yalnızca alt sınıf kodu ona *adıyla* erişemez. Kanıtı basittir: `Ogrenci` nesnesinin `maskeliTcNo()` çağrısı `private` olan `tcNo` alanını okuyup doğru sonuç üretmektedir.

3.3. Bir Alt Sınıfın Anatomisi

Aşağıdaki şema `Ogrenci` sınıfının dört kritik satırını işaretlemektedir. Bu dört öğeyi ayrı ayrı Bölüm 4'te ele alacağız; önce bütünü görmek yararlıdır.



Şekil 7.3 — `extends`, `super(...)`, `@Override` ve `super.metot()`

Üst sınıfın tam listelemesi aşağıdadır. Dikkat edilecek nokta: `Kisi` sınıfı mirasla ilgili **hiçbir şey yazmaz**. Bir sınıfın üst sınıf olabilmesi için özel bir bildirim gerekmez; `final` olmayan her sınıf türetilmeye açıktır.

Hafta_07/Java_Kodlari/ders/Kisi.java

```java
public class Kisi {

    // Ortak durum: her kiside vardir, turu ne olursa olsun.
    private final String ad;
    private final String tcNo;
    private final int    dogumYili;

    public static final int SIMDIKI_YIL = 2026;

    /* Ust sinifin yapicisi. Alt siniflar bu yapiciyi super(...) ile
     * cagirmak ZORUNDADIR; cunku parametresiz bir yapici yoktur. */
    public Kisi(String ad, String tcNo, int dogumYili) {
        System.out.println("    [Kisi yapicisi calisti]");
        if (ad == null || ad.isBlank()) {
            throw new IllegalArgumentException("Ad bos olamaz.");
        }
        if (tcNo == null || tcNo.length() != 11) {
            throw new IllegalArgumentException("TC No 11 haneli olmalidir.");
        }
        if (dogumYili < 1900 || dogumYili > SIMDIKI_YIL) {
            throw new IllegalArgumentException("Dogum yili gecersiz: " + dogumYili);
        }
        this.ad        = ad;
        this.tcNo      = tcNo;
        this.dogumYili = dogumYili;
    }

    // ---- Ortak davranis: alt siniflar bunlari MIRAS ALIR ----
    public String getAd()      { return ad; }
    public int    getDogumYili() { return dogumYili; }

    /** TC numarasinin yalnizca son iki hanesi disariya verilir. */
    public String maskeliTcNo() {
        return "*********" + tcNo.substring(9);
    }

}
```

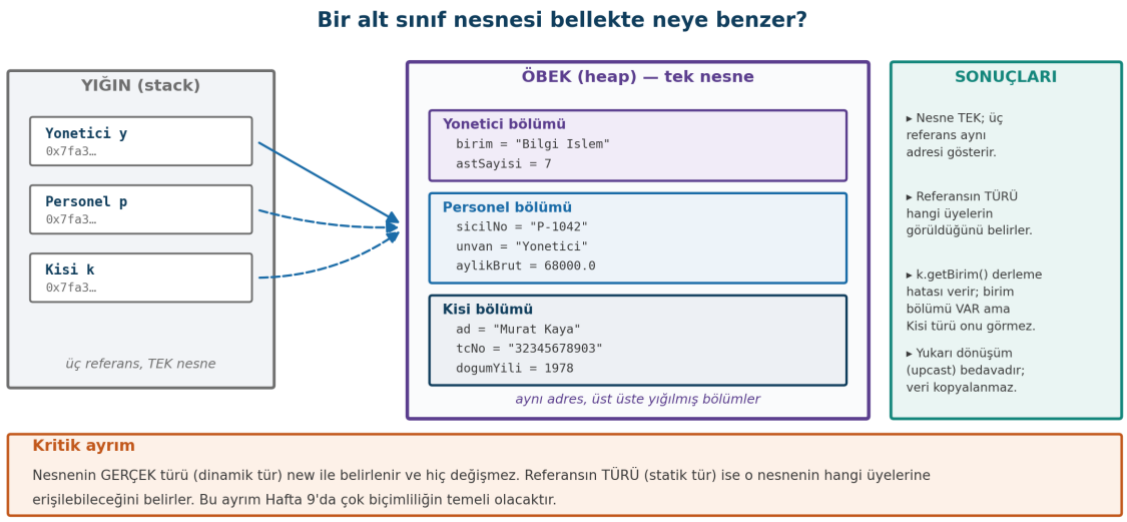
```
37      public int yas() {
38          return SIMDIKI_YIL - dogumYili;
39      }
40
41      public boolean resitMi() {
42          return yas() >= 18;
43      }
44
45      /** Alt siniflar bu metodu EZEREK (override) genisletecek. */
46      public String kimlikKarti() {
47          return ad + " | " + maskeliTcNo() + " | " + yas() + " yasinda";
48      }
```

Kod 7.4 — Üst sınıf `Kisi` — ortak durum ve ortak davranış

İki satır özellikle önemlidir. **12. satırdaki** yapıcı parametresizdir *değil*; üç parametre alır. Bu, alt sınıfları `super(...)` yazmaya **zorlar** (Bölüm 4.1). **46. satırdaki** `kimlikKarti()` metodu ise alt sınıflarca ezilmek üzere tasarlanmıştır; alt sınıflar bu metodu `super.kimlikKarti()` ile çağırıp üzerine ekleme yapacaktır.

3.4. Nesne Bellekte Nasıl Yerleşir?

Bir `Yonetici` nesnesi oluşturulduğunda bellekte üç ayrı nesne doğmaz; **tek** bir nesne doğar ve bu nesne üç sınıfın alanlarını üst üste yığılmış bölümler hâlinde taşır. Aynı nesneyi `Yonetici`, `Personel` veya `Kisi` türünde bir referansla tutabilirsiniz; değişen tek şey hangi üyeleri görebildiğinizdir.



Şekil 7.4 — Alt sınıf nesnesinin bellekteki katmanlı yerleşimi

Bu ayrımın kod üzerindeki karşılığı **yukarı dönüşüm** (upcast) denen işlemdir: alt tür bir nesne, üst tür bir referansa hiçbir dönüşüm sözdizimi yazmadan atanabilir.

Hafta_07/Java_Kodlari/ders/KisiDemo.java

```
1           System.out.println();
2           System.out.println("=== 7) Ust tur referansi alt tur nesnesini tutabilir ===");
3           Kisi ref = o;                       // yukari donusum (upcast) : guvenli
4           System.out.println("  ref.kimlikKarti() -> " + ref.kimlikKarti());
5           System.out.println("  ref.getClass()    -> " + ref.getClass().getSimpleName());
6           // ref.getGno();  // DERLEME HATASI: Kisi turunde getGno() yok.
```

Kod 7.5 — Yukarı dönüşüm ve statik türün getirdiği sınır

```
▸ TERMINAL — java KisiDemo (7. ve 8. bölüm)
=== 7) Ust tur referansi alt tur nesnesini tutabilir ===
  ref.kimlikKarti() -> Elif Demir | *********02 | 21 yasinda | OGRENCI YZM376014 (Yazilim
Muhendisligi)
  ref.getClass()    -> Ogrenci

=== 8) Uretilen ogrenci sayisi (Hafta 6 sayaci) ===
  Ogrenci.uretilen() -> 1
```

> **ⓘ BİLGİ**
>
> Yorum satırına alınmış `ref.getGno()` çağrısı derleme hatası verirdi: `The method getGno() is undefined for the type Kisi`. Nesnenin içinde `gno` alanı **vardır**, ama `Kisi` türündeki referans onu görmez. Referansın türü (statik tür) ile nesnenin gerçek türü (dinamik tür) arasındaki bu ayrım Hafta 9'un ana konusudur.

3.5. IS-A İlişkisinin Sınanması: `instanceof`

Bir nesnenin belirli bir türden olup olmadığı `instanceof` işleciyle sınanır. Miras zinciri boyunca yukarı doğru bütün türler için sonuç `true`'dur; aşağı doğru değildir.

```
▸ TERMINAL — java KisiDemo (6. bölüm — IS-A sınaması)
=== 6) IS-A iliskisinin sinanmasi ===
  o instanceof Ogrenci  -> true
  o instanceof Kisi     -> true
  o instanceof Object   -> true
  y instanceof Personel -> true
  y instanceof Kisi     -> true
  k instanceof Ogrenci  -> false   (Kisi her zaman Ogrenci DEGILDIR)
```

Son satır mirasın **tek yönlü** olduğunu gösterir: her `Ogrenci` bir `Kisi`dir, ama her `Kisi` bir `Ogrenci` değildir. Bu asimetri IS-A ilişkisinin tanımıdır ve Bölüm 4.8'deki karar kuralının temelini oluşturur.

> **▲ DİKKAT**
>
> `instanceof` bu haftada yalnızca IS-A ilişkisini **göstermek** için kullanılmaktadır. Gerçek programlarda `instanceof` zinciriyle tür ayırt etmek çoğu zaman kötü bir tasarımdır; Hafta 9'da bu zincirlerin çok biçimlilikle nasıl ortadan kaldırıldığını göreceğiz.

4. Derinleşme

4.1. Yapıcı Zinciri ve `super(...)`

Yapıcılar miras alınmaz. Bu kural ilk bakışta tuhaf görünür: `yas()` metodunu devralan `Ogrenci`, `Kisi(String, String, int)` yapıcısını neden devralmaz? Gerekçe anlamsal bir tutarlılıktır. Yapıcının işi **o sınıfın** nesnesini geçerli bir durumda kurmaktır; `Kisi` yapıcısı bir `Ogrenci` nesnesinin `ogrenciNo` alanını nasıl kuracağını bilemez.

Bunun yerine Java şu düzeni kurar: alt sınıfın yapıcısı, üst sınıfın yapıcısını `super(...)` ile açıkça çağırır ve **kendisinden önce çalıştırır**. Böylece nesnenin üst bölümü kurulmadan alt bölümüne dokunulmaz.

● *Hafta_07/Java_Kodlari/ders/Yonetici.java*

```java
    public Yonetici(String ad, String tcNo, int dogumYili, String sicilNo,
                    double aylikBrut, String birim, int astSayisi) {
        // Ust sinifin yapicisina unvani sabit gecirebiliriz.
        super(ad, tcNo, dogumYili, sicilNo, "Yonetici", aylikBrut);
        System.out.println("    [Yonetici yapicisi calisti]");
        this.birim     = birim;
        this.astSayisi = Math.max(0, astSayisi);
    }
```

Kod 7.6 — `Yonetici` yapıcısı — zincirin en alt halkası

Zincirin nasıl çalıştığını görmek için `YapiciZinciriDemo` programını yazdık. Bu program `A_Ust → B_Orta → C_Alt` üç seviyeli bir hiyerarşi kurar ve her adımda ekrana numaralı bir satır basar.

● *Hafta_07/Java_Kodlari/ders/YapiciZinciriDemo.java*

```java
class A_Ust {
    int alanA = yaz("1. A_Ust alani (alanA = 10)", 10);

    static { System.out.println("0a. A_Ust static blogu (sinif yuklenirken, bir kez)"); }

    { System.out.println("2. A_Ust baslatma blogu"); }

    A_Ust() {
        System.out.println("3. A_Ust() yapici govdesi");
    }

    static int yaz(String mesaj, int deger) {
        System.out.println(mesaj);
        return deger;
    }
}
```

Kod 7.7 — Üç seviyeli zincirin en üst halkası (`B_Orta` ve `C_Alt` aynı biçimdedir)

```
▸ TERMINAL — java YapiciZinciriDemo (birinci nesne)
--- new C_Alt() cagriliyor ---
0a. A_Ust static blogu (sinif yuklenirken, bir kez)
0b. B_Orta static blogu
0c. C_Alt static blogu
```

```
1. A_Ust alani (alanA = 10)
2. A_Ust baslatma blogu
3. A_Ust() yapici govdesi
4. B_Orta alani (alanB = 20)
5. B_Orta baslatma blogu
6. B_Orta() yapici govdesi
7. C_Alt alani (alanC = 30)
8. C_Alt baslatma blogu
9. C_Alt() yapici govdesi
--- nesne hazir: alanA=10 alanB=20 alanC=30 ---
```

Çıktıdaki sıra ezberlenecek değil, **türetilecek** bir sıradır. Numaralar şunu söyler: (a) `static` bloklar en başta, sınıf başına yalnızca bir kez ve **yukarıdan aşağıya** çalışır; (b) her seviyede önce alan başlatıcıları ve başlatma blokları, sonra yapıcı gövdesi çalışır; (c) seviyeler arasında **en üstten aşağıya** ilerlenir.



Şekil 7.5 — Yapıcı zinciri: çağrı yukarı çıkar, yürütme aşağı iner

Durum	Derleyicinin davranışı	Sonuç
`super(...)` yazıldı, yapıcının ilk deyimi	Kabul edilir	Doğru kullanım
`super(...)` yazıldı, ilk deyim değil	`Constructor call must be the first statement in a constructor`	Derleme hatası
`super(...)` yazılmadı, üst sınıfta parametresiz yapıcı var	Derleyici sessizce `super()` ekler	Çalışır
`super(...)` yazılmadı, üst sınıfta parametresiz yapıcı **yok**	`Implicit super constructor Kisi() is undefined`	Derleme hatası
Üst sınıfta hiç yapıcı yazılmadı	Varsayılan yapıcı vardır, `super()` çalışır	Çalışır
`this(...)` ve `super(...)` aynı yapıcıda	İkisi de ilk deyim olamaz	Derleme hatası

Tablo 7.5 — `super(...)` çağrısının kuralları ve derleyici iletileri

> ✓ İPUCU
>
> `this(...)` ile `super(...)` birlikte kullanılamaz, ama birbirini takip edebilir: alt sınıfın bir yapıcısı `this(...)` ile kendi başka yapıcısını çağırır, o yapıcı da `super(...)` ile üst sınıfa gider. Hafta 5'te öğrendiğimiz `this(...)` zinciri böylece miras zincirine bağlanır.

`static` bloklar yalnızca bir kez çalıştığı için ikinci nesnede sıra kısalır: `0a`, `0b` ve `0c` satırları bir daha görünmez, `1`–`9` satırları ise aynen yinelenir. Bu, Hafta 6'daki "sınıf yükleme tembeldir" kuralının miras altındaki görünümüdür. Yalnızca `new B_Orta()` çağrılırsa `7`, `8` ve `9` satırları hiç oluşmaz; zincir yalnızca o nesnenin bulunduğu seviyeye kadar işler.

4.2. Metot Ezme (Override) ve `@Override`

Alt sınıf, üst sınıftan devraldığı bir metodu **aynı imzayla** yeniden tanımlayarak davranışını değiştirebilir. Buna metot ezme denir. `Ogrenci` sınıfı `kimlikKarti()` metodunu ezmiş, ama üst sürümü çöpe atmamış; `super.kimlikKarti()` ile çağırıp üzerine eklemiştir.

● *Hafta_07/Java_Kodlari/ders/Ogrenci.java*

```
1       /* Ust siniftan gelen kimlikKarti() metodunu EZIYORUZ.
2        * super.kimlikKarti() ile ust surumu cagirip UZERINE ekliyoruz;
3        * boylece ortak kismi yeniden yazmiyoruz. */
4       @Override
5       public String kimlikKarti() {
6           return super.kimlikKarti() + " | OGRENCI " + ogrenciNo + " (" + bolum + ")";
7       }
```

Kod 7.8 — `kimlikKarti()` metodunun ezilmesi

Aynı metot üç seviyede de ezildiği için `Yonetici` nesnesinin kimlik kartı üç katmanın toplamıdır. Aşağıdaki çıktı bu birikimi göstermektedir: `Kisi` bölümü, `Personel` eki ve `Yonetici` eki tek satırda birleşmiştir.

```
▸ TERMINAL — java KisiDemo (5. bölüm — üç katmanın birleşimi)
=== 5) Iki seviyeli hiyerarsi ===
    [Kisi yapicisi calisti]
    [Personel yapicisi calisti]
    [Yonetici yapicisi calisti]
  Murat Kaya | *********03 | 48 yasinda | PERSONEL P-1042 (Yonetici) | Bilgi Islem birimi, 7 ast
  brut=68000.00  net=67320.00  tazminat=9520.00
  kidemYili() (Personel'den) -> 24
  yas() (Kisi'den)           -> 48
```

Ezmenin kuralları imza düzeyinde tanımlıdır. Aşağıdaki tablo neyin değişebileceğini, neyin değişemeyeceğini listeler.

Öğe	Ezmede durumu	Not
Metot adı	Aynı olmak zorunda	Farklıysa yeni metottur
Parametre listesi	Aynı olmak zorunda (sıra ve tür)	Farklıysa **aşırı yükleme** olur (Bölüm 6.2)

Öğe	Ezmede durumu	Not
Dönüş türü	Aynı ya da alt türü (covariant return)	`Kisi` yerine `Ogrenci` döndürülebilir
Erişim düzeyi	Daraltılamaz, genişletilebilir	`public` metot `protected` yapılamaz
`final` metot	Ezilemez	Derleme hatası (Bölüm 4.5)
Gövde	Tamamen değişebilir	Ezmenin asıl amacı budur
`@Override` etiketi	Zorunlu değil, **şart koşulmalı**	İmza hatasını derleme zamanında yakalar

Tablo 7.6 — Metot ezmede değişebilenler ve değişemeyenler

Bir üst sınıf, hangi metotlarının ezilmesini beklediğini tasarımıyla belirtebilir. `Hesap` sınıfı bunun temiz bir örneğidir: kesinti hesabının **oranını** üreten `protected` bir metot bırakmış, hesabın kendisini ise `public` bir metotta kapatmıştır.

Hafta_07/Java_Kodlari/ders/Hesap.java

```java
    /** Uzatma noktasi: alt siniflar yalnizca ORANI degistirir. */
    protected double kesintiOrani() {
        return 0.0;
    }

    public double aylikKesinti() {
        return bakiye * kesintiOrani();
    }
```

Kod 7.9 — Uzatma noktası (extension point) olarak `protected` metot

Hafta_07/Java_Kodlari/ders/VadeliHesap.java

```java
    /** Uzatma noktasi eziliyor; aylikKesinti() metodu YENIDEN YAZILMIYOR. */
    @Override
    protected double kesintiOrani() {
        return 0.005;
    }
```

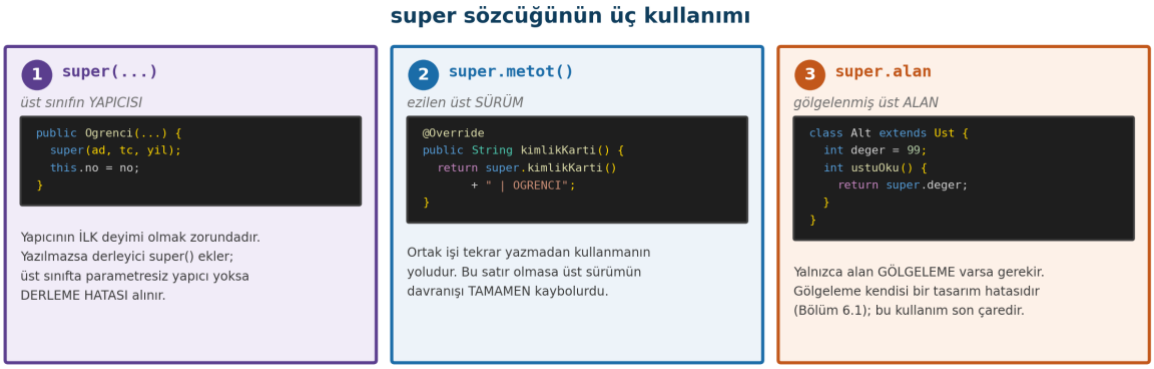
Kod 7.10 — Alt sınıf yalnızca uzatma noktasını ezer

```
▸ TERMINAL — java HesapDemo (4. bölüm — ezilen uzatma noktası)
=== 4) Ezilen uzatma noktasinin etkisi ===
  Hesap.aylikKesinti()       = 0.00
  VadeliHesap.aylikKesinti() = 99.60
  aylikKesinti() metodu VadeliHesap'ta YAZILMADI;
  yalnizca kesintiOrani() ezildi.
```

> **✓ İPUCU**
>
> Bu desende `aylikKesinti()` metodu `VadeliHesap` sınıfında **hiç yazılmamıştır**. Yalnızca tek satırlık `kesintiOrani()` ezilmiştir; sonuç yine de değişmiştir. İyi tasarlanmış bir üst sınıf, alt sınıflara mümkün olan **en küçük** uzatma noktasını bırakır.

4.3. `super` Sözcüğünün Üç Kullanımı

`super` sözcüğü üç ayrı işte kullanılır ve üçü birbirine karıştırılmamalıdır. İlk ikisi yaygın ve doğrudur; üçüncüsü ancak bir tasarım hatasının (alan gölgeleme) sonucunda gerekir.



Şekil 7.6 — `super(...)`, `super.metot()` ve `super.alan`

`super.metot()` çağrısının en değerli tarafı **kısıtlama ekleyerek genişletme**yi mümkün kılmasıdır. `VadeliHesap.cek()` metodu, üst sınıfın bakiye kontrolünü yeniden yazmak yerine kendi vade kontrolünü önüne koyup gerisini üst sınıfa devreder.

Hafta_07/Java_Kodlari/ders/VadeliHesap.java

```
1       /** Vade dolmadan cekim yapilamaz: ust sinif kuralina EK kisit. */
2       @Override
3       public boolean cek(double miktar) {
4           if (!vadeDoldumu()) {
5               System.out.println("  RET: vade dolmadi (" + gecenGun + "/" + vadeGunu + "
gun)");
6               return false;
7           }
8           return super.cek(miktar);       // temel kontroller ust sinifta kalir
9       }
```

Kod 7.11 — Üst sınıf kuralına ek kısıt koyarak ezme

```
▸ TERMINAL — java HesapDemo (2. ve 3. bölüm)
=== 2) Alt sinif : VadeliHesap ===
  TR1002 (Elif Demir) bakiye=20000.00 [vadeli 32 gun, gecen 0 gun]
  vade dolmadan cek(1000):
  RET: vade dolmadi (0/32 gun)
  sonuc -> false

=== 3) Vade doldu, ust sinif kurallari yine gecerli ===
  TR1002 (Elif Demir) bakiye=20000.00 [vadeli 32 gun, gecen 40 gun]
  faizIsle() = 920.55
  TR1002 (Elif Demir) bakiye=20920.55 [vadeli 32 gun, gecen 40 gun]
  cek(1000) -> true | TR1002 (Elif Demir) bakiye=19920.55 [vadeli 32 gun, gecen 40 gun]
  RET: yetersiz bakiye (19920.54794520548 < 999999.0)
  cek(999999) -> false   (ust siniftan gelen bakiye kontrolu)
```

Çıktının son iki satırı önemlidir: vade dolduktan sonra bile `cek(999999)` çağrısı reddedilmiştir. Bu reddi veren kod `VadeliHesap` içinde değil, `super.cek(...)` çağrısıyla ulaşılan `Hesap` sınıfındadır. Üst sınıfın kuralı **kaybolmamış**, üzerine eklenmiştir.

> **✕ SIK YAPILAN HATA**
>
> Ezilen bir metotta `super.metot()` çağrısını **unutmak**, üst sınıfın bütün doğrulama ve iş kurallarını sessizce devre dışı bırakır. Kod derlenir, program çalışır, ama artık yanlış çalışır. Bir metodu ezerken ilk sorulacak soru şudur: "Üst sınıfın yaptığı işi tamamen mi değiştiriyorum, yoksa üzerine mi ekliyorum?" İkinci durumda `super` çağrısı zorunludur.

4.4. `protected`: Mirasla Anlam Kazanan Erişim Düzeyi

Hafta 4'te dört erişim düzeyini tanıtmış, `protected`'ı "alt sınıflar konusunu görene kadar erteliyoruz" diyerek atlamıştık. Şimdi sırası geldi. `protected` bir üye, sınıfın kendisi, aynı paketteki sınıflar **ve farklı paketteki alt sınıflar** tarafından görülür; ilgisiz kodlar tarafından görülmez.



Şekil 7.7 — Dört erişim düzeyi ve `protected`'ın miras için anlamı

`Hesap` sınıfı üç erişim düzeyini bilinçli olarak yan yana kullanır: `hesapNo` alanı `private` (kimse dokunmasın), `bakiye` alanı `protected` (alt sınıflar faiz eklesin), `yatir()` ve `cek()` metotları `public` (herkes kullansın).

Hafta_07/Java_Kodlari/ders/Hesap.java

```
1  public class Hesap {
2
3      private final String hesapNo;          // disariya kapali, degismez
4      protected double bakiye;               // alt siniflar dogrudan gorebilir
5      protected final String sahip;
6
7      private static int sonrakiNo = 1001;
```

```
8
9       public Hesap(String sahip, double acilisBakiyesi) {
10          if (acilisBakiyesi < 0) {
11              throw new IllegalArgumentException("Acilis bakiyesi negatif olamaz.");
12          }
13          this.hesapNo = "TR" + sonrakiNo;
14          sonrakiNo++;
15          this.sahip   = sahip;
16          this.bakiye  = acilisBakiyesi;
17      }
18
19      /** final : hesap numarasi bicimi hicbir alt sinifta degistirilemez. */
20      public final String hesapNo() {
21          return hesapNo;
22      }
23
24      public double getBakiye() {
25          return bakiye;
26      }
```

Kod 7.12 — Üç erişim düzeyinin bir arada kullanımı

`VadeliHesap.faizIsle()` metodu `bakiye` alanına **doğrudan** eriştiği için `protected` gereklidir. Alan `private` olsaydı bu satır derlenmezdi ve üst sınıfa bir `bakiyeArtir(double)` metodu eklemek gerekirdi.

Hafta_07/Java_Kodlari/ders/VadeliHesap.java

```
1       /** VadeliHesap'a ozgu davranis: faiz isletme. */
2       public double faizIsle() {
3           double faiz = bakiye * YILLIK_FAIZ * gecenGun / 365.0;
4           bakiye += faiz;               // protected alana DOGRUDAN erisim
5           return faiz;
6       }
```

Kod 7.13 — `protected` alana alt sınıftan doğrudan erişim

▲ DİKKAT

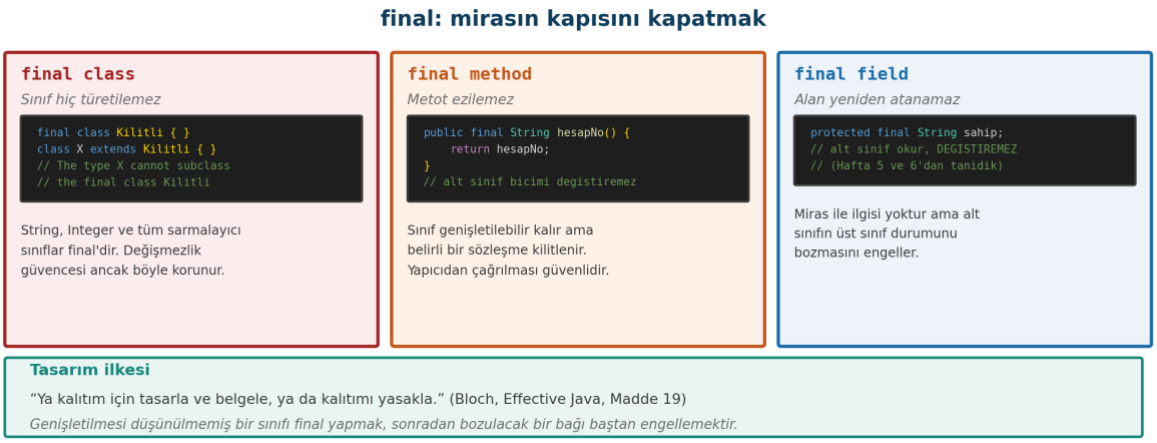
`protected` bir üye, `public` bir üye kadar bağlayıcı bir **sözleşmedir**. Bir kez `protected` yaptığınız alanı sonradan kaldırmak veya adını değiştirmek, o sınıftan türeyen bütün kodları bozar. Kural şudur: önce `private` dene; alt sınıf getter/setter ile yetiniyorsa `protected` yazma.

Soru	Yanıt `private` ise	Yanıt `protected` ise
Alt sınıf alana doğrudan yazacak mı?	Hayır — getter yeter	Evet — `protected` gerekli
Üst sınıf alanın temsilini sonra değiştirebilir mi?	Evet, serbestçe	Hayır, alt sınıflar bağlıdır
Alanın doğrulaması nerede kalır?	Setter içinde, tek noktada	Dağılır; alt sınıf atlayabilir
Kapsülleme (Hafta 4) korunuyor mu?	Tam olarak	Kısmen — alt sınıflara açılmış

Tablo 7.7 — `private` mı `protected` mı? Karar soruları

4.5. `final`: Mirasın Kapısını Kapatmak

`final` sözcüğünü Hafta 5 ve 6'da alanlar için kullandık: bir kez atanır, bir daha değişmez. Aynı sözcük sınıflara ve metotlara uygulandığında mirası kısıtlar.



Şekil 7.8 — `final` sınıf, `final` metot ve `final` alan

`Hesap` sınıfında `hesapNo()` metodu `final`dir. Bunun gerekçesi kurumsaldır: hesap numarası biçimi banka genelinde tektir ve hiçbir hesap türü onu değiştirmemelidir. Sınıfın kendisi `final` değildir; genişletilebilir kalmıştır.

Hafta_07/Java_Kodlari/ders/Hesap.java

```
1       /** final : hesap numarasi bicimi hicbir alt sinifta degistirilemez. */
2       public final String hesapNo() {
3           return hesapNo;
4       }
5
6       public double getBakiye() {
7           return bakiye;
8       }
```

Kod 7.14 — `final` metot ile kilitlenen sözleşme

```
▸ TERMINAL — java HesapDemo (5. bölüm — `final` metot)
=== 5) final metot : hesapNo() ===
  h.hesapNo() -> TR1001
  v.hesapNo() -> TR1002
  Bu metot final oldugundan hicbir alt sinif
  numara bicimini degistiremez.
```

Kullanım	Etkisi	Tipik gerekçe
`final class C`	`C`'den hiç türetilemez	Değişmezlik güvencesi (`String`, `Integer`)
`final` metot	Metot ezilemez	Sözleşmenin bozulmaması; yapıcıdan güvenli çağrı
`final` alan	Yeniden atanamaz	Kimlik bilgisi; alt sınıfın durumu bozmasını engelleme
Hiçbiri	Sınıf ve metotlar genişletilmeye	**Belgelenmiş** bir uzatma noktası varsa doğru

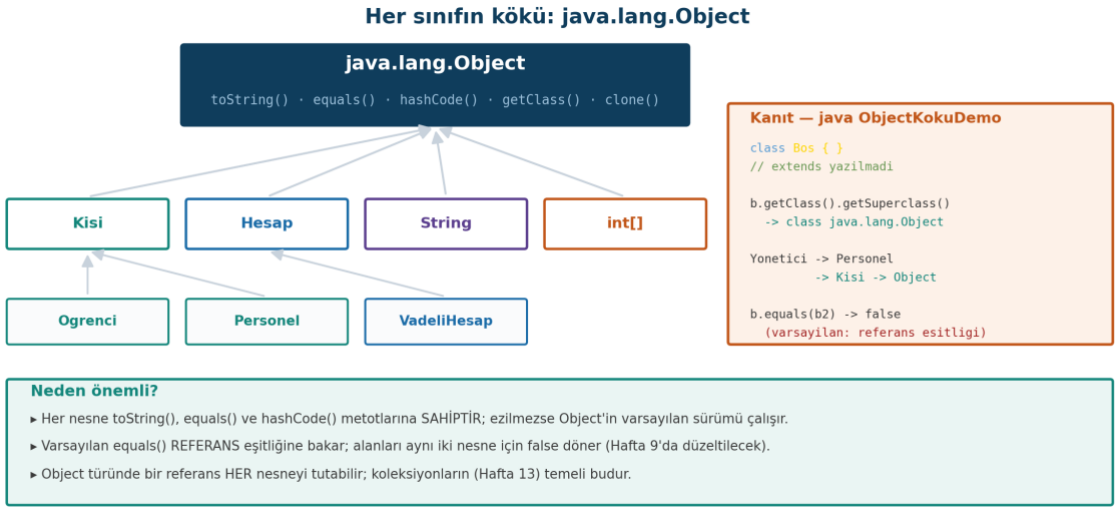
Kullanım	Etkisi	Tipik gerekçe
	açık	

Tablo 7.8 — `final` anahtar sözcüğünün üç bağlamı

> **✓ İPUCU**
>
> Bir sınıfın genişletilmeye açık olması bir **karar**dır, varsayılan değil. Joshua Bloch bu kararı şöyle özetler: "Ya kalıtım için tasarla ve belgele, ya da kalıtımı yasakla." Genişletilmesini düşünmediğiniz bir sınıfı `final` yapmak, sonradan bozulacak bir bağı baştan engellemektir.

4.6. Her Sınıfın Kökü: `Object`

Java'da `extends` yazmayan her sınıf, örtük olarak `java.lang.Object` sınıfından türer. Bu kuralın istisnası yoktur; diziler bile `Object`ten türer. Sonuç olarak Java'daki bütün sınıflar tek bir ağaç oluşturur.



Şekil 7.9 — `Object` kök ağacı ve `Object`ten devralınan metotlar

Bunu kanıtlamak için hiçbir üyesi olmayan bir sınıf yazdık ve üst sınıfını sorduk. Aynı program, `getSuperclass()` çağrısını `null` dönene kadar yineleyen kısa bir döngüyle `Yonetici` sınıfının miras zincirini de yukarı doğru izliyor.

Hafta_07/Java_Kodlari/ders/ObjectKokuDemo.java

```
    public static void main(String[] args) {
        System.out.println("=== 1) Her sinif Object'ten turer ===");
        Bos b = new Bos();
        System.out.println("  Bos sinifinda hicbir uye yazilmadi.");
        System.out.println("  b.getClass().getSimpleName() ->
" + b.getClass().getSimpleName());
        System.out.println("  b.getClass().getSuperclass() ->
" + b.getClass().getSuperclass());
        System.out.println("  b instanceof Object          -> " + (b instanceof Object));

        System.out.println();
```

```
10          System.out.println("=== 2) Miras zinciri yukari dogru izlenebilir ===");
11          Class<?> c = new Yonetici("Murat Kaya", "32345678903", 1978,
12                                    "P-2001", 55000.0, "Kalite", 3).getClass();
```

Kod 7.15 — `Object` kökünün kanıtı ve miras zincirinin izlenmesi

```
▸ TERMINAL — java ObjectKokuDemo (1. ve 2. bölüm)
=== 1) Her sinif Object'ten turer ===
  Bos sinifinda hicbir uye yazilmadi.
  b.getClass().getSimpleName() -> Bos
  b.getClass().getSuperclass() -> class java.lang.Object
  b instanceof Object          -> true

=== 2) Miras zinciri yukari dogru izlenebilir ===
    [Kisi yapicisi calisti]
    [Personel yapicisi calisti]
    [Yonetici yapicisi calisti]
  Yonetici -> Personel -> Kisi -> Object
```

`Object` sınıfından devralınan metotlar, ezilmedikleri sürece varsayılan davranışlarını sürdürür. İkisi bu hafta önemlidir.

`Object` metodu	Varsayılan davranış	Ne zaman ezilir?
`toString()`	`SinifAdi@onaltilikHash` biçiminde metin	Neredeyse **her zaman** — hata ayıklamayı kolaylaştırır
`equals(Object)`	Referans eşitliği (`==` ile aynı)	Değer eşitliği gerektiğinde (Hafta 9)
`hashCode()`	Nesneye özgü bir tam sayı	`equals()` ezildiğinde **zorunlu** (Hafta 13)
`getClass()`	Nesnenin gerçek türünü verir	`final`dir, ezilemez
`clone()`	Alanları sığ kopyalar (`protected`)	Hafta 5'teki kopya yapıcı genellikle daha iyidir

Tablo 7.9 — `Object`ten devralınan metotlar ve varsayılan davranışları

```
▸ TERMINAL — java ObjectKokuDemo (3. ve 4. bölüm)
=== 3) Object'ten gelen varsayilan davranis ===
  b.equals(b)  -> true   (varsayilan: referans esitligi)
  b.equals(b2) -> false  (alanlar ayni olsa da FALSE)
  toString() ezilmezse sinifAdi@hashCode basilir:
  b.toString() bicimi -> Bos@<onaltilik hash>

=== 4) toString() ezildiginde ===
    [Kisi yapicisi calisti]
    [Ogrenci yapicisi calisti]
  o.toString() -> Ogrenci{Elif Demir, YZM376014, GNO=0.0}
```

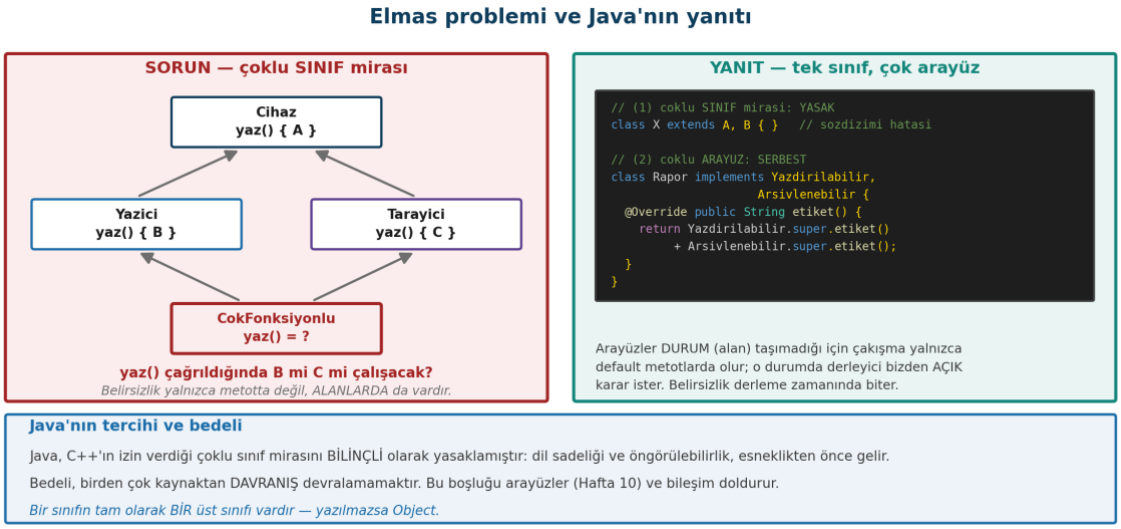
Çıktının ilk bölümü varsayılan `equals()` davranışını gösterir: iki `Bos` nesnesinin hiçbir alanı olmadığı hâlde `equals` sonucu `false`'tur, çünkü karşılaştırılan şey içerik değil **adrestir**. İkinci bölümde ise ezilmiş `toString()` çalışmış ve okunabilir bir metin üretmiştir.

> **✓ İPUCU**
>
> Yazdığınız her sınıfta `toString()` metodunu ezin. Maliyeti üç satırdır; kazancı, hata ayıklarken `System.out.println(nesne)` yazdığınızda `Ogrenci@7a81197d` yerine `Ogrenci{Elif Demir, YZM376014, GNO=0.0}` görmenizdir.

4.7. Çoklu Miras ve Elmas Problemi

Bazı diller (örneğin C++) bir sınıfın birden çok sınıftan türemesine izin verir. Java vermez. Bu kısıtlamanın gerekçesi **elmas problemi** (diamond problem) olarak bilinen belirsizliktir.

Şu senaryoyu düşünün: `Cihaz` sınıfının bir `yaz()` metodu var. `Yazici` ve `Tarayici` sınıfları bu metodu farklı biçimlerde eziyor. Şimdi hem yazıcı hem tarayıcı olan bir `CokFonksiyonlu` sınıfı ikisinden birden türese, `yaz()` çağrıldığında hangi gövde çalışacaktı? Belirsizlik yalnızca metotlarda değil, **alanlarda** da vardır: aynı alan iki yoldan devralındığında bir mi iki mi kopya olacağı da tanımsızdır.



Şekil 7.10 — Elmas problemi ve Java'nın yanıtı

Java bu belirsizliği kaynağında keser: bir sınıfın tam olarak **bir** doğrudan üst sınıfı olabilir. Buna karşılık bir sınıf istediği kadar **arayüz** gerçekleyebilir; çünkü arayüzler durum (alan) taşımaz. Java 8'den sonra arayüzler `default` gövdeli metot içerebildiği için orada da bir çakışma olasılığı doğar, ama bu çakışma derleyici tarafından yakalanır ve programcıdan açık bir karar istenir.

Hafta_07/Java_Kodlari/ders/ObjectKokuDemo.java

```java
interface Yazdirilabilir {
    String yazdir();
    default String etiket() { return "[yazdirilabilir]"; }
}

interface Arsivlenebilir {
    String arsivAnahtari();
    default String etiket() { return "[arsivlenebilir]"; }
```

```
 9  }
10
11  class Rapor implements Yazdirilabilir, Arsivlenebilir {
12      private final String konu;
13
14      Rapor(String konu) { this.konu = konu; }
15
16      public String yazdir()       { return "RAPOR: " + konu; }
17      public String arsivAnahtari() { return "ARS-" + konu.hashCode(); }
18
19      /* Iki arayuzden ayni adli default metot geldigi icin derleyici
20       * bizden acik bir karar ister; hangi surumu istedigimizi
21       * Arayuz.super.metot() ile yazariz. */
22      @Override
23      public String etiket() {
24          return Yazdirilabilir.super.etiket() + Arsivlenebilir.super.etiket();
25      }
26  }
```

Kod 7.16 — Çoklu arayüz gerçeklemesi ve `default` metot çakışmasının çözümü

```
▸ TERMINAL — java ObjectKokuDemo (6. bölüm — elmas probleminin çözümü)
=== 6) Elmas problemi : coklu arayuz serbest ===
  r.yazdir()        -> RAPOR: Donem Sonu
  r.arsivAnahtari() -> ARS-1789881144
  r.etiket()        -> [yazdirilabilir][arsivlenebilir]
  Iki arayuzden ayni adli default metot geldi;
  derleyici acik karar istedi, Arayuz.super ile verdik.
  'class X extends A, B' ise SOZDIZIMI HATASIDIR.
```

> **ⓘ BİLGİ**
>
> Arayüzler Hafta 10'un konusudur; burada yalnızca elmas probleminin Java'daki çözümünü görmek için kullanıldı. Şimdilik akılda tutulacak tek şey şudur: **sınıf mirası tekildir, arayüz gerçeklemesi çoğuldur.**

Dil / mekanizma	Çoklu miras	Belirsizlik nasıl çözülür?
C++ — çoklu sınıf mirası	Var	Sanal miras (`virtual`) ve nitelikli çağrı ile programcı çözer
Java — çoklu **sınıf** mirası	Yok	Dil düzeyinde yasak; sorun hiç doğmaz
Java — çoklu **arayüz** gerçeklemesi	Var	Arayüz durum taşımaz; `default` çakışması derleyici tarafından yakalanır
Java — bileşim (composition)	Sınırsız	Her bağımlılık ayrı bir alandır; çakışma olmaz

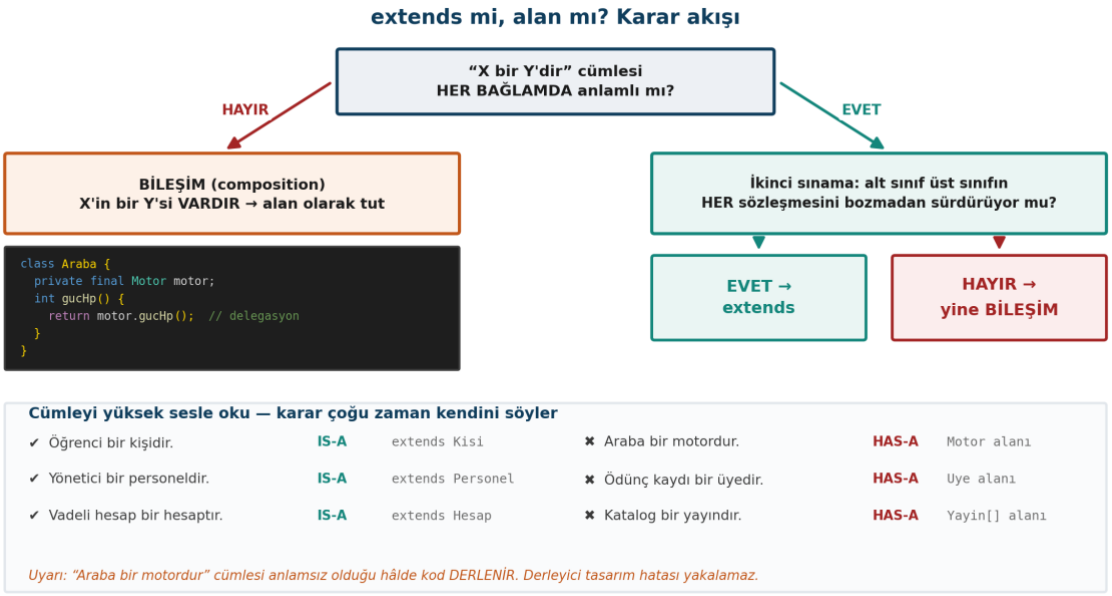
Tablo 7.10 — Çoklu miras yaklaşımlarının karşılaştırması

4.8. IS-A mı HAS-A mı? Kalıtım ile Bileşim Arasındaki Karar

Bu bölüm haftanın en önemli bölümüdür. Sözdizimi öğrenilmesi kolay, karar verilmesi zordur. Yanlış karar derleme hatası vermez; yıllarca taşınan bir tasarım borcu bırakır.

HAS-A ilişkisi (bileşim, composition) Bir sınıfın başka bir sınıfın nesnesini **alan olarak** tutmasıdır. "Arabanın bir motoru vardır" cümlesi bu ilişkiyi tanımlar. Bileşimde sarmalayan sınıf, sarmalanan nesnenin hangi yeteneklerini dışarıya açacağına kendisi karar verir.

Karar kuralı tek cümledir: **"X bir Y'dir" cümlesi her bağlamda anlamlıysa** `extends`, **"X'in bir Y'si vardır" anlamlıysa** alan. Aşağıdaki akış bu kuralı ikinci bir sınamayla birlikte gösterir.



Şekil 7.11 — `extends` mi, alan mı? İki aşamalı karar akışı

Yanlış kararın somut bedelini görmek için `BilesimDemo` programında bilinçli olarak hatalı bir tasarım yazdık: `KotuAraba extends Motor`. Cümleyi yüksek sesle okuyalım — "Araba bir motordur." Anlamsızdır; buna karşın kod hatasız derlenir.

Hafta_07/Java_Kodlari/ders/BilesimDemo.java

```java
/* YANLIS TASARIM -------------------------------------------------
 * "Arabanin motoru var" demek yerine "araba bir motordur" dedik.
 * Sonuc: yakitPompasiTest() gibi SERVIS ici bir ayrinti arabanin
 * genel arayuzune sizdi ve araba iki motorlu olamaz hale geldi. */
class KotuAraba extends Motor {
    private final String plaka;

    KotuAraba(String plaka, int gucHp) {
        super(gucHp);
        this.plaka = plaka;
    }

    String plaka() { return plaka; }
}
```

Kod 7.17 — Bilinçli olarak yanlış tasarım: `KotuAraba extends Motor`

```
▸ TERMINAL — java BilesimDemo (1. bölüm — yanlış tasarımın sonuçları)
=== 1) YANLIS: KotuAraba extends Motor ===
    Motor calisti (130 HP)
  Sorun: asagidaki iki satir DERLENIYOR ve calisiyor.
```

```
    [servis] yakit pompasi testi
  kotu instanceof Motor -> true   ('Araba bir motordur' anlamsizdir)
  Ayrica bu araba IKI motorlu olamaz;
  cunku motoru bir ALAN degil, KENDISIDIR.
```

Üç somut zarar oluştu. Birincisi, `Motor` sınıfının servis içi ayrıntısı olan `yakitPompasiTest()` metodu arabanın genel arayüzüne **sızdı**. İkincisi, `kotu instanceof Motor` sınaması `true` döndü; artık bir araba, motor beklenen her yere geçirilebilir. Üçüncüsü ve en ağırı: bu araba **iki motorlu olamaz**, çünkü motoru bir alan değil, kendisidir.

Doğrusu bileşimdir. `Araba` sınıfı `Motor` nesnesini bir alanda tutar ve motorun hangi yeteneklerini dışarıya açacağına kendisi karar verir. Bu aktarma işine **delegasyon** denir.

● *Hafta_07/Java_Kodlari/ders/BilesimDemo.java*

```java
class Araba {
    private final String plaka;
    private final Motor motor;          // HAS-A
    protected int kmSayaci;

    Araba(String plaka, Motor motor) {
        this.plaka = plaka;
        this.motor = motor;
    }

    String plaka()   { return plaka; }
    int gucHp()      { return motor.gucHp(); }        // delegasyon

    void kontagiAc() {
        System.out.println("  " + plaka + ": kontak acildi");
        motor.calistir();                             // delegasyon
    }
```

Kod 7.18 — Doğru tasarım: `Araba` HAS-A `Motor` (delegasyon)

```
▸ TERMINAL — java BilesimDemo (2. bölüm — bileşim ile doğru tasarım)
=== 2) DOGRU: Araba HAS-A Motor ===
  61 CD 002: kontak acildi
    Motor calisti (130 HP)
  61 CD 002: 120 km surudu (toplam 120 km)
    Motor durdu
  61 CD 002: kontak kapandi
  61 CD 002: motor kapali, surulemez
  61 CD 002 | 130 HP | 120 km
  Motor'un yakitPompasiTest() metodu ARABA'nin
  arayuzunde YOKTUR; Araba neyi acacagini secti.
```

Miras her zaman yanlış değildir elbette. “Elektrikli araba bir arabadır” cümlesi anlamlıdır; burada `extends` doğru seçimdir. Aşağıdaki alt sınıf hem `super.surus(...)` ile üst sınıfın işini yeniden kullanır hem de kendi kısıtını ekler.

● *Hafta_07/Java_Kodlari/ders/BilesimDemo.java*

```java
class ElektrikliAraba extends Araba {
    private int sarjYuzdesi;

    ElektrikliAraba(String plaka, Motor motor, int sarjYuzdesi) {
        super(plaka, motor);
        this.sarjYuzdesi = Math.min(100, Math.max(0, sarjYuzdesi));
    }

    @Override
    void surus(int km) {
        int gereken = Math.max(1, km / 5);
        if (sarjYuzdesi < gereken) {
            System.out.println("  " + plaka() + ": sarj yetersiz ("
                               + sarjYuzdesi + "%)");
            return;
        }
        super.surus(km);                // ust sinifin isini tekrar yazmiyoruz
```

Kod 7.19 — Doğru kalıtım: `ElektrikliAraba` IS-A `Araba`

```
▸ TERMINAL — java BilesimDemo (3. bölüm — doğru kalıtım)
=== 3) DOGRU KALITIM: ElektrikliAraba IS-A Araba ===
  61 EF 003: kontak acildi
    Motor calisti (210 HP)
  61 EF 003: 100 km surudu (toplam 100 km)
  61 EF 003: kalan sarj 10%
  61 EF 003: sarj yetersiz (10%)
  61 EF 003 | 210 HP | 100 km | sarj 10%
```

Ölçüt	Kalıtım (`extends`)	Bileşim (alan)
Kurduğu ilişki	IS-A — tür akrabalığı	HAS-A — sahiplik
Bağ sıkılığı	Sıkı; üst sınıf değişimi alt sınıfı etkiler	Gevşek; yalnızca kullanılan arayüz önemli
Kaç kaynaktan yararlanılır?	Yalnızca bir üst sınıftan	İstenildiği kadar alandan
Sarmalananın iç ayrıntısı	Dışarıya sızar	Sarmalayan sınıf neyi açacağını seçer
Çalışma zamanında değişebilir mi?	Hayır, tür sabittir	Evet, alan yeni bir nesneyle değiştirilebilir
Çok biçimlilik (Hafta 9) sağlar mı?	Evet, doğrudan	Yalnızca arayüzle birlikte (Hafta 10)
Ne zaman tercih edilir?	Tür akrabalığı gerçekten varsa	Şüphe varsa — **varsayılan tercih budur**

Tablo 7.11 — Kalıtım ile bileşimin karşılaştırması

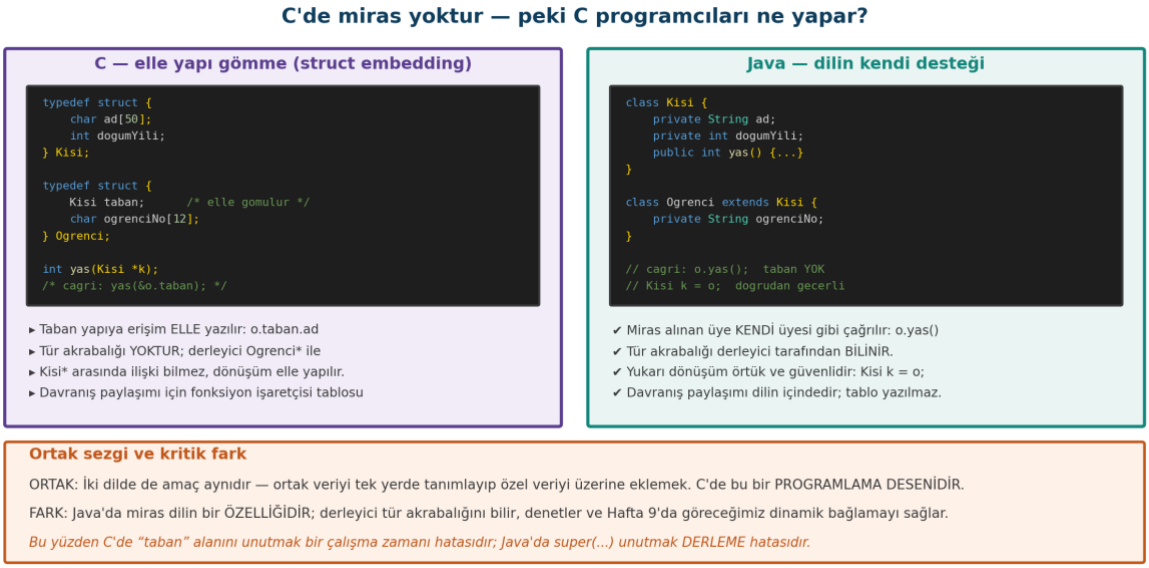
> **▲ DİKKAT**
>
> Bir tasarım sınavı sorusunda "`extends` mi bileşim mi?" sorulduğunda yanıtı gerekçesiz vermeyin. Beklenen yanıt şudur: (a) cümleyi kurun ("X bir Y'dir" / "X'in bir Y'si vardır"), (b) alt sınıfın üst sınıfın **her** sözleşmesini bozmadan sürdürüp sürdüremediğini sınayın, (c) kararı bu iki sınamaya dayandırın.

5. C'den Java'ya Köprü

⇄ C DİLİNDEN JAVA'YA KÖPRÜ

C dilinde miras diye bir mekanizma **yoktur**. Ama C programcıları aynı ihtiyacı elle karşılar: ortak alanları bir `struct` içine koyup onu diğer yapının **ilk alanı** olarak gömerler. Java'daki miras, bu programlama deseninin dil düzeyine taşınmış ve derleyici tarafından denetlenen hâlidir.

- C'de gömülü yapıya erişim **elle** yazılır: `o.taban.ad`. Java'da miras alınan üye kendi üyesi gibi çağrılır: `o.getAd()`.
- C'de `Ogrenci*` ile `Kisi*` arasında derleyicinin bildiği bir ilişki yoktur; dönüşüm elle ve tür denetimsiz yapılır. Java'da `Kisi k = o;` atamasını derleyici doğrular.
- C'de ortak davranışı paylaşmanın yolu fonksiyon işaretçisi tablosu kurmaktır. Java'da bu tablo (sanal metot tablosu) JVM tarafından otomatik oluşturulur.
- C'de "taban" alanını yapı bildiriminde ilk sıraya koymayı unutmak, çalışma zamanında bozuk bellek erişimine yol açar. Java'da `super(...)` çağrısını unutmak **derleme hatasıdır**.



Şekil 7.12 — C'de yapı gömme deseni ile Java mirasının karşılaştırması

Konu	C dili	Java
Ortak alanları paylaşma	`struct` gömme (elle desen)	`extends` (dil özelliği)
Ortak alana erişim	`o.taban.ad`	`o.getAd()` — doğrudan
Tür akrabalığı	Yok; `void*` ve elle dönüşüm	Var; derleyici denetler
Yukarı dönüşüm	`(Kisi *) &o` — denetimsiz	`Kisi k = o;` — örtük ve güvenli
Ortak davranış paylaşımı	Fonksiyon işaretçisi tablosu (elle)	Metot ezme + dinamik bağlama (JVM)
Görünürlük denetimi	`static` ile dosya kapsamı	`private` / paket / `protected` / `public`
Hatanın yakalandığı an	Çalışma zamanı (çoğu zaman hiç)	Derleme zamanı
Genişletmeyi yasaklama	Dil desteği yok	`final class` / `final` metot

Tablo 7.12 — C'de yapı gömme ile Java mirasının karşılaştırması

> **ⓘ BİLGİ**
>
> C++ bilenler için bir not: Java'nın mirası C++'ın `public` mirasına karşılık gelir. C++'ın `private` ve `protected` miras biçimleri, çoklu sınıf mirası ve sanal miras (`virtual` inheritance) Java'da **yoktur**. Java bu sadeleştirmeyi bilinçli olarak yapmıştır.

6. Sık Yapılan Hatalar

Bu bölümdeki hataların ilk üçü **derleme hatası vermez**. Kod derlenir, program çalışır, yalnızca yanlış çalışır. Bu yüzden ezberden değil, çalıştırıp gözleyerek öğrenilmeleri gerekir. `MirasHatalariDemo` programı üçünü bir arada gösterir.



Şekil 7.13 — Derleyicinin yakalamadığı üç miras hatası

6.1. Alan Gölgeleme (Field Hiding)

Alt sınıfta üst sınıftaki bir alanla **aynı adı** kullanmak, üst alanı silmez; onu gölgeler. Sonuçta tek nesnede aynı adı taşıyan iki ayrı alan bulunur. Hangisine erişildiği, nesnenin gerçek türüne değil, **referansın statik türüne** göre belirlenir.

Hafta_07/Java_Kodlari/ders/MirasHatalariDemo.java

```
1  class UstSayac {
2      int deger = 10;
3      int ustDegerOku() { return deger; }
4  }
5
6  class AltSayac extends UstSayac {
7      int deger = 99;              // AYNI AD -> ust alan golgelenir, silinmez
```

```
    int altDegerOku() { return deger; }
    int ustuAcikOku() { return super.deger; }
}
```

Kod 7.20 — Alan gölgeleme: tek nesnede iki `deger` alanı

```
▸ TERMINAL — java MirasHatalariDemo (HATA 1)
=== HATA 1 : alan golgeleme ===
  a.altDegerOku()  -> 99   (alt siniftaki deger)
  a.ustDegerOku()  -> 10   (ust siniftaki deger, DEGISMEDI)
  a.ustuAcikOku()  -> 10   (super.deger)
  ((UstSayac) a).deger -> 10
  a.deger              -> 99
  Tek nesnede IKI ayri 'deger' alani vardir.
  Alanlara erisim STATIK TURE gore cozulur.
```

✕ SIK YAPILAN HATA

Aynı nesne, aynı alan adı, **iki farklı değer**. Bu davranış hiçbir yerde işe yaramaz; alan gölgeleme daima bir kazadır. Çözüm basittir: alt sınıfta üst sınıftaki bir alan adını tekrar kullanmayın. Farklı bir değer gerekiyorsa farklı bir ad ya da `protected` bir setter kullanın.

6.2. Ezme Yerine Aşırı Yükleme

İmzanın en küçük farkı, ezmeyi aşırı yüklemeye dönüştürür. Aşağıdaki örnekte parametre türü `Object` yerine `String` yazıldığı için üst sürüm **ezilmemiş**, yanına yeni bir metot eklenmiştir. Sonuçta aynı nesne, aynı argümanla çağrıldığında referansın türüne göre farklı yanıt verir.



Şekil 7.14 — Ezme (override) ile aşırı yükleme (overload) karşılaştırması

Hafta_07/Java_Kodlari/ders/MirasHatalariDemo.java

```
class Bicimleyici {
    String bicimle(Object o) { return "UST(Object): " + o; }
}
```

```
 4
 5  class SayiBicimleyici extends Bicimleyici {
 6      /* Parametre turu String; ust siniftaki imza Object.
 7       * Bu bir EZME degil, ASIRI YUKLEMEDIR. @Override yazilsa
 8       * derleyici "must override or implement a supertype method"
 9       * hatasi verirdi. */
10      String bicimle(String s) { return "ALT(String): " + s; }
11  }
```

Kod 7.21 — Ezme sanılan aşırı yükleme

```
▸ TERMINAL — java MirasHatalariDemo (HATA 2)
=== HATA 2 : ezme yerine asiri yukleme ===
  sb.bicimle("42")          -> ALT(String): 42
  ((Bicimleyici) sb).bicimle("42") -> UST(Object): 42
  Ayni nesne, ayni arguman, FARKLI sonuc.
  Metot ezilmedi; yeni bir metot eklendi.
  Cozum: her ezme denemesine @Override yaz.
```

> **✓ İPUCU**
>
> Bu hatanın tek ve kesin çözümü `@Override` etiketidir. Etiket yazıldığında derleyici imzanın gerçekten bir üst sınıf metoduyla eşleştiğini denetler; eşleşmezse `The method f(String) of type Alt must override or implement a supertype method` hatası verir. Kural: **her ezme denemesine `@Override` yazın.**

6.3. Yapıcıdan Ezilebilir Metot Çağırmak

Üst sınıfın yapıcısı, alt sınıfın alanları henüz atanmadan çalışır (Bölüm 4.1). Bu yüzden üst sınıf yapıcısının çağırdığı bir metot alt sınıfta ezilmişse, o metot **yarı kurulmuş** bir nesne üzerinde çalışır.

● *Hafta_07/Java_Kodlari/ders/MirasHatalariDemo.java*

```
 1  class UstRapor {
 2      UstRapor() {
 3          System.out.println("    UstRapor yapicisi -> baslik(): " + baslik());
 4      }
 5      String baslik() { return "genel rapor"; }
 6  }
 7
 8  class AltRapor extends UstRapor {
 9      private String konu = "donem sonu";      // super() BITTIKTEN sonra atanir
10
11      AltRapor() {
12          super();
13          System.out.println("    AltRapor yapicisi -> baslik(): " + baslik());
14      }
15
16      @Override
17      String baslik() {
18          return "rapor: " + konu;             // ilk cagrida konu HENUZ null
19      }
```

Kod 7.22 — Yapıcıdan ezilebilir metot çağırmanın sonucu

```
▸ TERMINAL — java MirasHatalariDemo (HATA 3)
=== HATA 3 : yapicidan ezilebilir metot cagirma ===
  new AltRapor() cagriliyor:
    UstRapor yapicisi -> baslik(): rapor: null
    AltRapor yapicisi -> baslik(): rapor: donem sonu
  nesne hazir  -> baslik(): rapor: donem sonu
  Ilk satirda konu alani null'di; cunku ust sinifin
  yapicisi, alt sinifin alanlari atanmadan once
  ezilmis metodu cagirdi.
```

İlk satırda `konu` alanı `null`dur; çünkü `super()` çağrısı bittiğinde alt sınıfın alan başlatıcıları henüz çalışmamıştır. İkinci satırda aynı metot doğru sonucu verir. Aynı metot, aynı nesne, **iki farklı sonuç**.

> **✕ SIK YAPILAN HATA**
>
> Kural: bir yapıcıdan yalnızca `private`, `static` veya `final` metotlar çağrılmalıdır. Bunların hiçbiri ezilemeyeceği için yarı kurulmuş nesne sorunu doğmaz. `Hesap.hesapNo()` metodunun `final` olmasının ikinci gerekçesi de budur.

6.4. Kalıtımın Kötüye Kullanımı

Bu, bu haftanın en pahalı hatasıdır ve Bölüm 4.8'de ayrıntılı incelendi. Belirtisi tektir: `extends` yazma gerekçesi olarak "üst sınıfın metotlarını hazır almak" gösteriliyorsa karar yanlıştır. Aşağıdaki tablo yaygın yanlış eşleştirmeleri ve düzeltmelerini içerir.

Yazılan	Cümle sınaması	Doğrusu
`class Katalog extends Yayin`	"Katalog bir yayındır" — anlamsız	`Katalog` içinde `Yayin[] raf` alanı
`class OduncKaydi extends Uye`	"Ödünç kaydı bir üyedir" — anlamsız	`OduncKaydi` içinde `Uye uye` alanı
`class Araba extends Motor`	"Araba bir motordur" — anlamsız	`Araba` içinde `Motor motor` alanı
`class Ogrenci extends Adres`	"Öğrenci bir adrestir" — anlamsız	`Ogrenci` içinde `Adres adres` alanı
`class Kare extends Dikdortgen`	"Kare bir dikdörtgendir" — geometride doğru, **kodda tehlikeli**	Ayrı `Sekil` soyutlaması (Hafta 10)
`class Yonetici extends Personel`	"Yönetici bir personeldir" — **doğru**	`extends` yerinde kullanılmış

Tablo 7.13 — Kalıtımın yanlış ve doğru kullanıldığı durumlar

> **ⓘ BİLGİ**
>
> Tablodaki `Kare extends Dikdortgen` satırı ünlü bir tasarım tartışmasıdır. Matematikte her kare bir dikdörtgendir; ama `Dikdortgen` sınıfının `genislikAyarla()` ve `yukseklikAyarla()` metotlarını `Kare` bozmadan sürdüremez (birini değiştirmek diğerini de değiştirmek zorundadır). IS-A cümlesinin doğru olması yetmez; alt sınıf üst sınıfın **her sözleşmesini** sürdürebilmelidir.

6.5. Derleyici İletileri Sözlüğü

Bu haftanın hataları belirli derleyici iletileri üretir. İletiyi okumayı öğrenmek, hatayı tahmin etmekten hızlıdır.

Derleyici iletisi	Nedeni	Çözüm
`Implicit super constructor Kisi() is undefined`	Üst sınıfta parametresiz yapıcı yok, `super(...)` yazılmadı	Yapıcının ilk satırına `super(...)` ekleyin
`Constructor call must be the first statement in a constructor`	`super(...)` ilk deyim değil	Çağrıyı yapıcının en başına taşıyın
`The method f(String) of type Alt must override or implement a supertype method`	`@Override` yazıldı ama imza uyuşmuyor	İmzayı üst sınıftakine eşitleyin
`The type X cannot subclass the final class Kilitli`	`final` sınıftan türetme denemesi	Bileşim kullanın
`The method getGno() is undefined for the type Kisi`	Üst tür referansla alt tür üyesine erişim	Referans türünü değiştirin veya aşağı dönüşüm yapın
`Syntax error on token ","` (`extends A, B`)	Çoklu sınıf mirası denemesi	Tek üst sınıf + arayüz(ler) kullanın

Tablo 7.14 — Mirasla ilgili derleyici iletileri ve çözümleri

7. İyi Tasarım İlkeleri

Aşağıdaki ilkeler bu haftanın kodlarında uygulanmıştır; laboratuvarda da uygulanması beklenir.

1. **Bileşimi varsayılan tercih yap.** Kalıtım, IS-A ilişkisi gerçekten varsa ve alt sınıf üst sınıfın her sözleşmesini sürdürebiliyorsa kullanılır. Şüphe hâlinde alan kullan.
2. **Hiyerarşiyi sığ tut.** Üç seviyeden derin hiyerarşiler okunmayı ve hata ayıklamayı hızla zorlaştırır. `Yayin → Dergi → AkademikDergi` üst sınırdır.
3. **Her ezme denemesine `@Override` yaz.** Etiketin çalışma zamanı maliyeti yoktur; karşılığında bütün bir hata sınıfını derleme zamanına taşır.
4. **Uzatma noktalarını küçük tut ve belgele.** Alt sınıfın ezmesi beklenen metot mümkün olan en küçük parça olsun (`kesintiOrani()` gibi), tüm hesap değil.
5. **Önce `private`, gerekiyorsa `protected`.** `protected` bir üye `public` kadar bağlayıcı bir sözleşmedir; geri alınamaz.
6. **Yapıcıdan ezilebilir metot çağırma.** Yalnızca `private`, `static` veya `final` metot çağır.
7. **Genişletilmesini istemediğin sınıfı `final` yap.** Kararı varsayılana bırakmak, sonradan bozulacak bir bağ üretir.
8. **`toString()` metodunu daima ez.** Üç satırlık bu ekleme, hata ayıklama süresini gözle görülür biçimde kısaltır.
9. **Alt sınıfta üst sınıf alan adını tekrar kullanma.** Alan gölgeleme her zaman bir kazadır.
10. **`super.metot()` çağrısını bilinçli olarak ver ya da verme.** Vermeme kararı, üst sınıfın bütün kurallarını devre dışı bırakmak anlamına gelir; bunu bilerek yap.

Kontrol sorusu	Beklenen yanıt
Bu `extends` bildiriminin IS-A cümlesi nedir?	Yüksek sesle okunduğunda anlamlı bir cümle
Alt sınıf üst sınıfın hangi sözleşmesini bozuyor?	Hiçbirini
Hiyerarşi kaç seviye derin?	En çok üç
Ezilen her metotta `@Override` var mı?	Evet, istisnasız
`protected` yaptığım her üye gerçekten gerekli mi?	Evet; `private` yetmiyor
Yapıcıdan çağrılan metotlar ezilebilir mi?	Hayır
Bu sınıf genişletilecek mi?	Evet ise belgelenmiş, hayır ise `final`

Tablo 7.15 — Kod teslim etmeden önceki tasarım kontrolü

8. Bir Sonraki Haftaya Köprü

Bu hafta bir alt sınıf nesnesinin üst tür referansla tutulabildiğini gördük: `Kisi ref = o;` satırı geçerliydi. Ama şu soruyu yanıtlamadık: `ref.kimlikKarti()` çağrıldığında hangi gövde çalışır — `Kisi` sınıfındaki mi, `Ogrenci` sınıfındaki ezilmiş mi?

Yanıtı zaten çıktılarda gördünüz: ezilmiş sürüm çalıştı. Referansın türü `Kisi` olmasına rağmen nesnenin gerçek türü `Ogrenci` olduğundan `Ogrenci.kimlikKarti()` seçildi. Bu davranışa **dinamik bağlama** denir ve Hafta 9'un konusudur.

Aynı şekilde `Gorev5_Kutuphane` görevinde `Katalog` sınıfı, elindeki nesnenin `Kitap`, `Dergi` yoksa `AkademikDergi` olduğunu bilmeden her birinin kendi ödünç kuralını işletmeyi başardı. Bu, çok biçimliliğin (polymorphism) somut faydasıdır.

Bu hafta öğrenilen	Hafta 9–10'da genişleyecek biçimi
Üst tür referansı alt tür nesnesini tutar	Çok biçimli diziler ve döngüler; `switch` yığınlarının kaldırılması
Metot ezme (override)	Dinamik bağlama; sanal metot tablosu; `toString`/`equals` sözleşmeleri
`instanceof` ile tür sınaması	Güvenli aşağı dönüşüm (downcast) ve desen eşleme
Uzatma noktası olarak `protected` metot	`abstract` metot — gövdesi olmayan, ezilmesi **zorunlu** uzatma noktası
Çoklu sınıf mirasının yasak olması	`interface` ile çoklu davranış sözleşmesi ve `default` metotlar
"Kalıtım yerine bileşim"	Arayüze programlama; Hafta 14'te Strategy deseni

Tablo 7.16 — Bu haftanın kavramlarının Hafta 9 ve 10'daki karşılıkları

9. Haftanın Özeti

- **Miras**, bir sınıfın başka bir sınıfın üyelerini devralmasıdır ve `extends` ile kurulur. Asıl değeri kod tekrarını azaltmak değil, **tür akrabalığı** (IS-A) kurmaktır.

- Alt sınıf üst sınıfın `public` ve `protected` üyelerini devralır; `private` üyeler nesnede vardır ama adıyla erişilemez. **Yapıcılar miras alınmaz.**
- `super(...)` üst sınıf yapıcısını çağırır ve yapıcının **ilk deyimi** olmak zorundadır. Yazılmazsa derleyici `super()` ekler; üst sınıfta parametresiz yapıcı yoksa bu derleme hatasıdır.
- Yapıcı zinciri **en tepeden aşağıya** yürütülür: her seviyede önce alan başlatıcıları ve başlatma blokları, sonra yapıcı gövdesi.
- **Metot ezme** aynı imzayla yeniden tanımlamaktır; `@Override` etiketi imza hatasını derleme zamanında yakalar. `super.metot()` ezilen üst sürümü yeniden kullanır.
- `protected` erişim düzeyi, üyeyi alt sınıflara açar ama dünyaya kapatır. `public` kadar bağlayıcı bir sözleşmedir; önce `private` denenmelidir.
- `final` sınıf türetilemez, `final` metot ezilemez. Genişletilmesi düşünülmeyen sınıflar `final` yapılmalıdır.
- `extends` yazmayan her sınıf **`java.lang.Object`**'ten türer; `toString()`, `equals()` ve `hashCode()` metotları buradan gelir.
- Java **çoklu sınıf mirasını yasaklar** (elmas problemi); çoklu arayüz gerçeklemesine izin verir, çünkü arayüzler durum taşımaz.
- Karar kuralı: "X bir Y'dir" anlamlıysa `extends`, "X'in bir Y'si vardır" anlamlıysa **alan** (bileşim). Şüphe hâlinde bileşim.
- Derleyicinin yakalamadığı üç hata: **alan gölgeleme**, **ezme yerine aşırı yükleme** ve **yapıcıdan ezilebilir metot çağırma**.

▶ BİR SONRAKİ HAFTA

Hafta 8 **ara sınav** haftasıdır; yeni konu işlenmez. Hafta 9'da **çok biçimlilik** (polymorphism) ile bu haftanın kurduğu hiyerarşiler çalışır hâle gelecek: üst tür bir referansla çağrılan ezilmiş metodun neden alt sınıf sürümünü çalıştırdığını (dinamik bağlama), çok biçimli dizilerle nasıl `switch` yığınlarından kurtulduğumuzu ve `toString`, `equals`, `hashCode` metotlarının doğru nasıl ezildiğini göreceğiz. Bu haftanın `Kisi`, `Hesap` ve `Yayin` hiyerarşileri Hafta 9'da aynen kullanılacaktır.

10. Alıştırmalar

Zorluk göstergesi: ●○○ temel · ●●○ orta · ●●● ileri. İlk dört alıştırma kavram, sonraki üçü kod okuma ve hata bulma, kalan beşi programlamadır.

10.1. Kavram Soruları

Alıştırma 1 ●○○

Bir alt sınıfın üst sınıftan **devralmadığı** üç şeyi yazınız. Her biri için neden devralınmadığını bir cümleyle gerekçelendiriniz. Ardından şu iddiayı değerlendiriniz: "`private` alanlar miras alınmadığı için alt sınıf nesnesinde bulunmazlar." İddia doğru mu? Yanıtınızı `maskeliTcNo()` metodunun çalışmasıyla ilişkilendirerek açıklayınız.

İpucu: Bölüm 3.2'deki tabloyu ve altındaki kırmızı kutuyu okuyunuz.

Alıştırma 2 ●●○

`super(...)` çağrısının yapıcının **ilk deyimi** olmak zorunda olmasının nedenini, nesne başlatma sırasına (Hafta 5, Bölüm 4.1 ve bu hafta Bölüm 4.1) dayanarak açıklayınız. Bu kural olmasaydı hangi somut sorun ortaya çıkardı? Yanıtınızda "yarı kurulmuş nesne" ifadesini kullanınız ve Bölüm 6.3'teki hatayla bağlantısını kurunuz.

Alıştırma 3 ●●○

Java'nın çoklu sınıf mirasını yasaklamasını **elmas problemi** üzerinden anlatınız. Ardından şu soruyu yanıtlayınız: Java çoklu arayüz gerçeklemesine izin verirken aynı belirsizlik neden doğmuyor? Java 8'den sonra `default` metotlarla kısmen doğan çakışma nasıl çözülüyor?

İpucu: Bölüm 4.7 ve `ObjectKokuDemo.java` dosyasının `Rapor` sınıfı.

Alıştırma 4 ●●○

Aşağıdaki altı cümle için IS-A mı HAS-A mı olduğunu belirleyiniz ve karşılığında `extends` mi alan mı kullanılacağını yazınız: (a) Kamyon bir araçtır. (b) Bilgisayarın bir işlemcisi vardır. (c) Vadeli hesap bir hesaptır. (d) Sipariş bir müşteridir. (e) Kitaplığın kitapları vardır. (f) Kare bir dikdörtgendir. Son madde için ek olarak, IS-A cümlesi doğru olmasına rağmen kalıtımın neden sorunlu olabileceğini açıklayınız.

İpucu: Bölüm 4.8'deki karar akışı ve Bölüm 6.4'teki mavi bilgi kutusu.

10.2. Kod Okuma ve Hata Bulma

Alıştırma 5 ve 6, aşağıdaki iki kısa listelemeye dayanır.

alistirma_5_6.java

```java
// Alistirma 5 icin
class A { int d = 1; int oku() { return d; } }
class B extends A { int d = 2; int oku() { return d; } }

// Alistirma 6 icin
class Ust { void yaz(Object o) { System.out.println("UST"); } }
class Alt extends Ust { void yaz(String s) { System.out.println("ALT"); } }
```

Kod 7.23 — Alıştırma 5 ve 6'nın kod listelemeleri

Alıştırma 5 ●●●

Yukarıdaki ilk listelemeye göre `A a = new B();` satırından sonra `a.d`, `a.oku()` ve `((B) a).d` ifadelerinin her birinin ne döndüğünü **çalıştırmadan** tahmin ediniz, sonra yazıp doğrulayınız. Üç sonucun farklı olmasının nedenini, alan erişimi ile metot çağrısının **hangi türe göre** çözüldüğü ayrımına dayandırarak açıklayınız. Tahmininiz yanlışsa hangi kuralı gözden kaçırdığınızı yazınız.

İpucu: Bölüm 6.1 ve `MirasHatalariDemo.java` dosyasının HATA 1 bölümü.

Alıştırma 6 ●●○

Yukarıdaki ikinci listelemede ezme niyetiyle yazılmış ama ezmeyen bir metot vardır. Hatayı bulunuz, `@Override` etiketi eklenirse derleyicinin vereceği iletiyi yazınız ve iki farklı düzeltme yolu öneriniz. Ardından `Ust u = new Alt(); u.yaz("abc");` çağrısının hangi gövdeyi çalıştıracağını gerekçesiyle belirtiniz.

Alıştırma 7 ●●●

Bir öğrenci `VadeliHesap.cek(double)` metodunu ezerken `super.cek(miktar)` çağrısını yazmayı unutmuş ve gövdeyi şöyle bırakmıştır: vade dolmuşsa `bakiye -= miktar; return true;`. Bu değişikliğin yol açtığı **üç** somut hatayı yazınız ve her biri için hangi çıktının değişeceğini `HesapDemo` programının 3. bölümüne bakarak belirtiniz.

İpucu: Bölüm 4.3'teki kırmızı kutu ve `Hesap.cek()` metodunun içindeki iki kontrol.

10.3. Programlama Alıştırmaları

Alıştırma 8 ●●○

`Kisi` hiyerarşisine `Ziyaretci` adlı yeni bir alt sınıf ekleyiniz. Sınıfın `ziyaretNedeni` ve `gorusulenBirim` alanları bulunsun; `kimlikKarti()` metodunu `super` çağrısıyla ezsin; `gecerliMi()` adlı bir metot ziyaretçinin reşit olup olmadığını `resitMi()` üzerinden denetlesin. Ardından `Kisi[]` türünde bir dizi kurup içine bir `Ogrenci`, bir `Personel`, bir `Yonetici` ve bir `Ziyaretci` yerleştirerek hepsinin kimlik kartını tek döngüde yazdırınız.

Alıştırma 9 ●●●

`Hesap` hiyerarşisine `KrediliHesap` adlı bir alt sınıf ekleyiniz. Bu hesap türü bakiyenin **eksiye düşmesine** belirli bir limite kadar izin vermelidir. `cek(double)` metodunu ezerken üst sınıfın "yetersiz bakiye" kuralını genişletmeniz gerekecek; bunu `super.cek(...)` çağrısını **koruyarak** yapamayacağınızı fark edeceksiniz. Çözümünüzü iki biçimde yazınız: (a) `Hesap` sınıfına yeni bir `protected` uzatma noktası ekleyerek, (b) `Hesap` sınıfına dokunmadan. İki çözümü sürdürülebilirlik açısından karşılaştırınız.

İpucu: Bölüm 4.2'deki `kesintiOrani()` deseni birinci çözümün modelidir; `alinabilirEnFazla()` gibi bir uzatma noktası düşünebilirsiniz.

Alıştırma 10 ●●○

Bölüm 4.8'deki `KotuAraba extends Motor` tasarımını bileşime dönüştüren `IyiAraba` sınıfını sıfırdan yazınız. Sınıf iki motorlu araçları da desteklemelidir (`Motor onMotor`, `Motor arkaMotor`). `toplamGuc()` metodu iki motorun gücünü toplasın; `kontagiAc()` metodu her iki motoru çalıştırsın. `Motor` sınıfının `yakitPompasiTest()` metodunun `IyiAraba` arayüzünde **görünmediğini** bir yorum satırıyla belirtiniz.

Alıştırma 11 ●○○

`Object` sınıfının `toString()` metodunu ezmeyen bir sınıf yazıp bir nesnesini `System.out.println` ile yazdırınız. Çıktıdaki `@` işaretinden sonraki değerin ne olduğunu araştırıp yazınız. Ardından `toString()` metodunu ezip aynı satırı tekrar çalıştırınız. Son olarak `equals()` metodunu **ezmeden** iki eşit içerikli nesne oluşturup karşılaştırınız; sonucun neden `false` olduğunu açıklayınız.

İpucu: `ObjectKokuDemo.java` dosyasının 3. ve 4. bölümleri beklenen çıktı biçimini gösterir.

Alıştırma 12 ●●●

Üç seviyeli bir hiyerarşi kurarak yapıcı zincirini **deneyle** doğrulayınız: `Ucak → YolcuUcagi → GenisGovdeliUcak`. Her sınıfın bir `static` bloğu, bir alan başlatıcısı, bir başlatma bloğu ve bir yapıcısı olsun; her biri ekrana kendi adını bassın. İki nesne üretip çıktıları karşılaştırınız. Sonucu bir tablo hâlinde sununuz: hangi satırlar iki kez, hangileri yalnızca bir kez göründü ve neden?

İpucu: `YapiciZinciriDemo.java` dosyası birebir bu deneyin modelidir; sınıf adlarını ve mesajları değiştirmeniz yeterlidir.

11. Öz Değerlendirme

Aşağıdaki maddelerin tamamını işaretleyebiliyorsanız bu haftanın kazanımlarına ulaşmışsınız demektir. İşaretleyemediğiniz her madde için karşısındaki bölüme dönünüz.

Öz Değerlendirme Kontrol Listesi

- ☐ Bir sınıfı `extends` ile türetebiliyor, alt sınıfın hangi üyeleri devraldığını söyleyebiliyorum. *(Bölüm 3.1, 3.2)*
- ☐ IS-A ile HAS-A ilişkisini ayırt edip bir tasarım kararını gerekçelendirebiliyorum. *(Bölüm 4.8, Şekil 7.8)*
- ☐ `super(...)` çağrısının ilk deyim olma zorunluluğunu nesne başlatma sırasıyla açıklayabiliyorum. *(Bölüm 4.1, Şekil 7.4)*
- ☐ Yapıcı zincirinin çıktısını çalıştırmadan tahmin edebiliyorum. *(Bölüm 4.1)*
- ☐ Bir metodu ezip `super.metot()` ile üst sürümü yeniden kullanabiliyorum. *(Bölüm 4.2, 4.3)*
- ☐ Ezme ile aşırı yüklemeyi imza düzeyinde ayırt edebiliyor ve `@Override` etiketinin ne yakaladığını biliyorum. *(Bölüm 4.2, 6.2, Şekil 7.11)*
- ☐ `protected` erişim düzeyinin ne zaman gerektiğine karar verebiliyorum. *(Bölüm 4.4, Şekil 7.7)*
- ☐ `final` sınıf ve `final` metodun mirası nasıl kısıtladığını açıklayabiliyorum. *(Bölüm 4.5, Şekil 7.14)*
- ☐ Her sınıfın `Object`ten türediğini kanıtlayabiliyor, varsayılan `toString()` ve `equals()` davranışını biliyorum. *(Bölüm 4.6, Şekil 7.10)*
- ☐ Elmas problemini anlatabiliyor ve Java'nın çözümünü özetleyebiliyorum. *(Bölüm 4.7, Şekil 7.9)*

☐ Alan gölgeleme, yanlış aşırı yükleme ve yapıcıdan ezilebilir metot çağırma hatalarını tanıyorum. *(Bölüm 6.1–6.3, Şekil 7.12)*

☐ Bir `extends` bildiriminin yerinde olup olmadığını cümle sınamasıyla denetleyebiliyorum. *(Bölüm 6.4, 7)*

12. Kaynaklar

12.1. Ders Kitapları

Kaynak	İlgili bölüm	Not
Deitel & Deitel, *Java How to Program* (11. baskı)	Bölüm 9 — Inheritance (9.3 `protected`, 9.4 hiyerarşi, 9.5 `Object`)	Temel düzey, adım adım
Horstmann, *Core Java Volume I* (12. baskı)	Bölüm 5.1 — Classes, Superclasses, Subclasses; 5.2 — Object	Yapıcı zinciri için en açık anlatım
Schildt, *Java: The Complete Reference* (12. baskı)	Bölüm 8 — Inheritance (`super`, çok seviyeli hiyerarşi, `final`)	Başvuru kaynağı
Bloch, *Effective Java* (3. baskı)	Madde 18, 19 ve 12 (bileşim, kalıtım tasarımı, `toString`)	Haftanın tasarım felsefesi

Tablo 7.17 — Bu hafta için önerilen okumalar

12.2. Çevrimiçi Kaynaklar

- **Oracle Java Tutorials — Inheritance:** `extends`, miras alınan üyeler ve `Object` kökünün temel anlatımı.
- **Oracle Java Tutorials — Overriding and Hiding Methods:** ezme ile gizleme (hiding) arasındaki fark ve `@Override` kullanımı.
- **Oracle Java Tutorials — Using the Keyword `super`:** `super(...)` ve `super.metot()` kullanımları, örtük `super()` kuralı.
- **Java Language Specification, §8.1.4 — Superclasses and Subclasses:** miras ilişkisinin biçimsel tanımı (ileri düzey).
- **Java Language Specification, §8.4.8 — Inheritance, Overriding, and Hiding:** ezme kurallarının tam listesi.
- **Java Language Specification, §12.5 — Creation of New Class Instances:** yapıcı zincirinin biçimsel tanımı.

12.3. İletişim ve Destek

Konu	Sorumlu	Kanal
Teori, kavram soruları	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Ders saati sonrası · e-posta
Laboratuvar, kod hataları	Arş. Gör. Betül MUMCU	Laboratuvar saatleri · e-posta
Teslim ve puanlama	Arş. Gör. Betül MUMCU	Ders yönetim sistemi
Ortam kurulumu (JDK, VS Code)	Laboratuvar sorumlusu	Hafta 1 föyü, Görev 0

Tablo 7.18 — Soru ve destek için başvuru noktaları